Chapter 6
Link Layer

A note on the use of these ppt slides:

We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a lot of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

Modified by Man Hon Cheung

Chapter 6: Link layer

our goals:

- understand principles behind link layer services:
  - error detection, correction
  - sharing a broadcast channel: multiple access
  - link layer addressing
  - local area networks: Ethernet, VLANs
- instantiation, implementation of various link layer technologies

Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs
- addressing, ARP
- Ethernet
- switches
- VLANS

6.5 link virtualization: MPLS

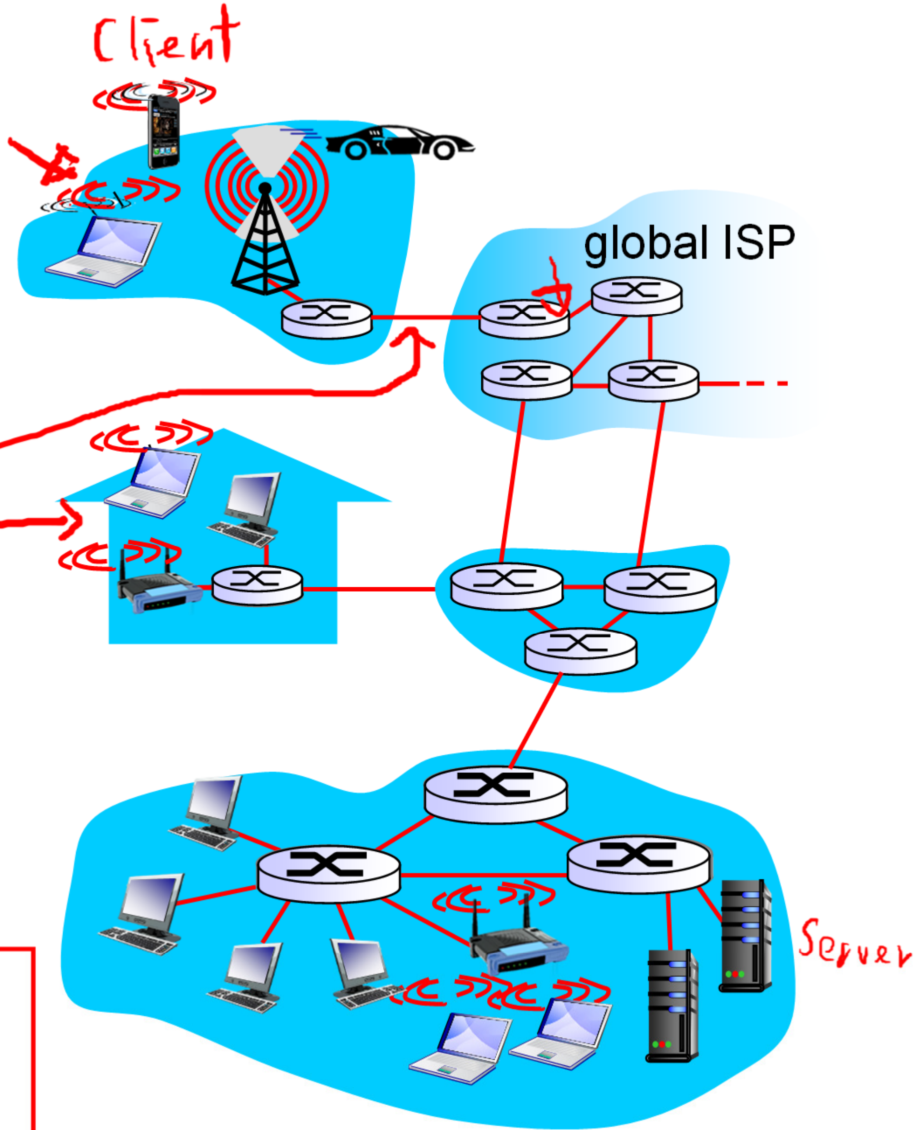
6.6 data center networking

6.7 a day in the life of a web request

Link layer: introduction

terminology:

- hosts and routers: nodes
- communication channels that connect adjacent nodes along communication path: links
  - wired links
  - wireless links
  - LANs
- layer-2 packet: *frame,* encapsulates datagram



data-link layer has responsibility of transferring datagram from one node to *physically adjacent* node over a link

1-hop

Link layer: context

- datagram transferred by different link protocols over different links:
  - e.g., Ethernet on first link, frame relay on intermediate links, 802.11 on last link
- each link protocol provides different services
  - e.g., may or may not provide rdt over link

transportation analogy:

- trip from Princeton to Lausanne
  - limo: Princeton to JFK
  - plane: JFK to Geneva
  - train: Geneva to Lausanne
- tourist = datagram
- travel agent = routing algorithm
- trip segment = communication link
- transportation mode = link layer protocol

Link layer services

- *framing:*
  - encapsulate datagram into frame, adding header, trailer
- *link access:* MAC protocol
  - channel access in shared medium
  - "MAC" addresses used in frame headers to identify source, dest
    - different from IP address!
- *reliable delivery between adjacent nodes:*
  - we learned how to do this already (chapter 3)!
  - seldom used on low bit-error link (fiber, some twisted pair)
  - wireless links: high error rates

Link layer services (more)

- *flow control:*
  - pacing between adjacent sending and receiving nodes
- *error detection:*
  - errors caused by signal attenuation, noise.
  - receiver detects presence of errors:
    - signals sender for retransmission or drops frame
- *error correction:*
  - receiver identifies *and corrects* bit error(s) without resorting to retransmission
- *half-duplex and full-duplex*
  - with half duplex, nodes at both ends of link can transmit, but not at same time

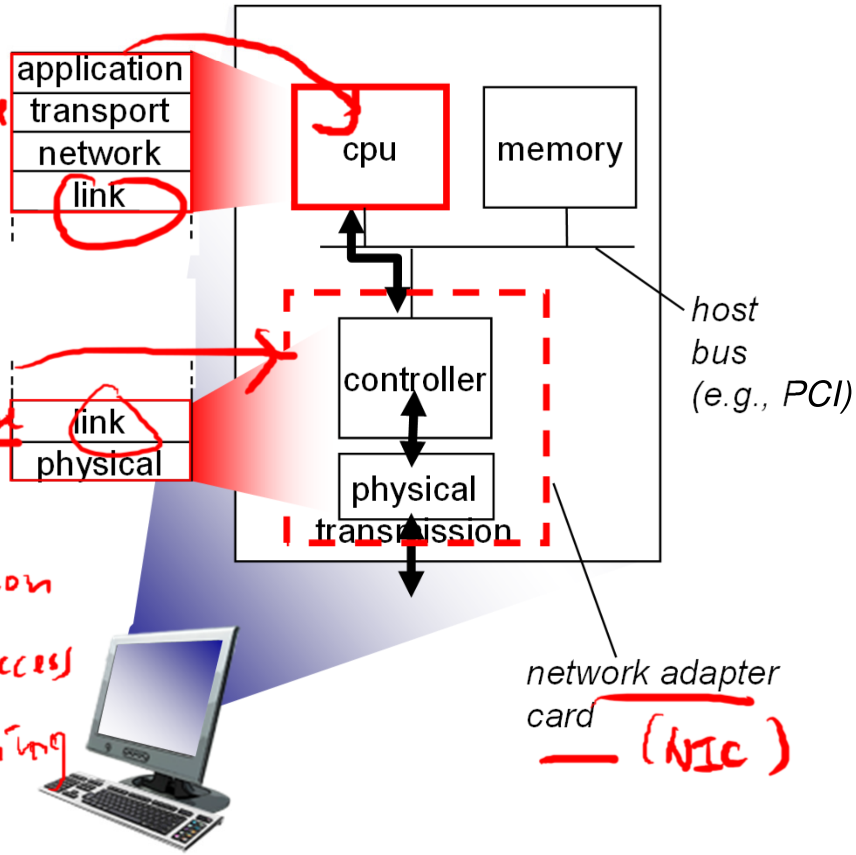
Where is the link layer implemented?

- in each and every host
- link layer implemented in "adaptor" (aka *network interface card* NIC) or on a chip
  - Ethernet card, 802.11 card; Ethernet chipset
  - implements link, physical layer
- attaches into host's system buses
- combination of hardware, software, firmware

· Assemble addr info
· pass datagram to netw layer

Software

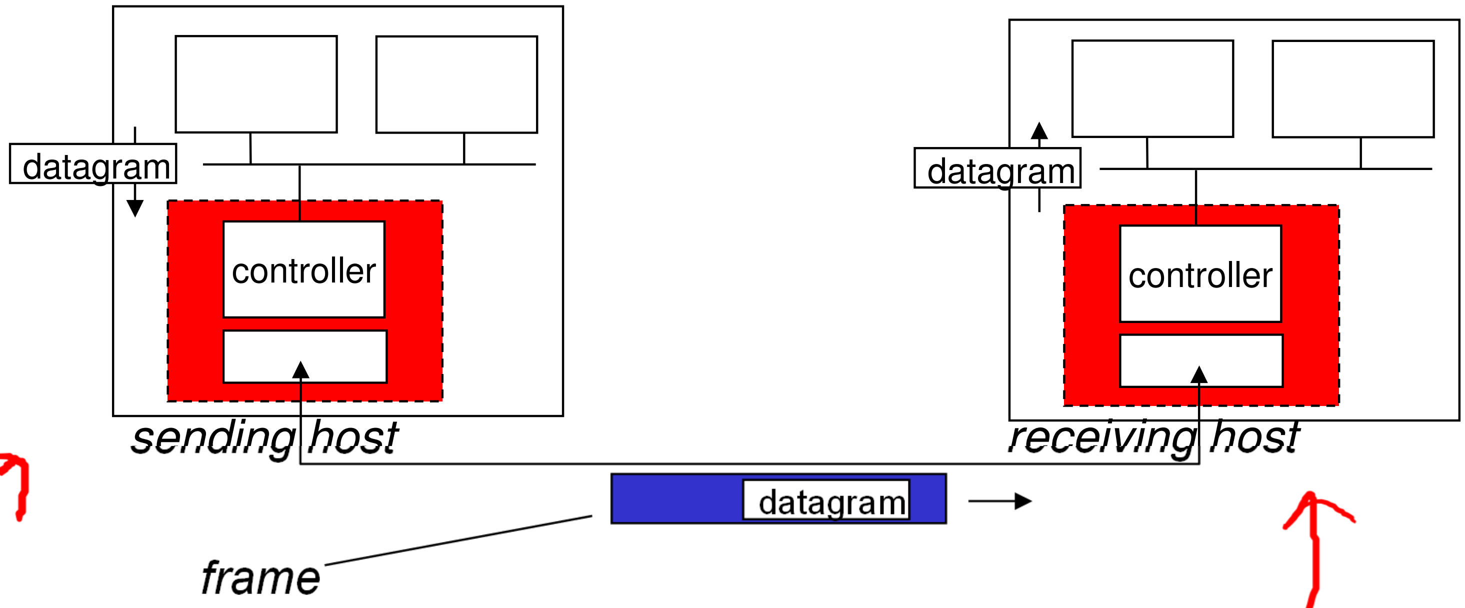
hardware



(NIC)

· error detection
· link access
· framing

Adaptors communicating



- sending side:
  - encapsulates datagram in frame
  - adds error checking bits, rdt, flow control, etc.
- receiving side
  - looks for errors, rdt, flow control, etc
  - extracts datagram, passes to upper layer at receiving side

Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs
- addressing, ARP
- Ethernet
- switches
- VLANS

6.5 link virtualization: MPLS

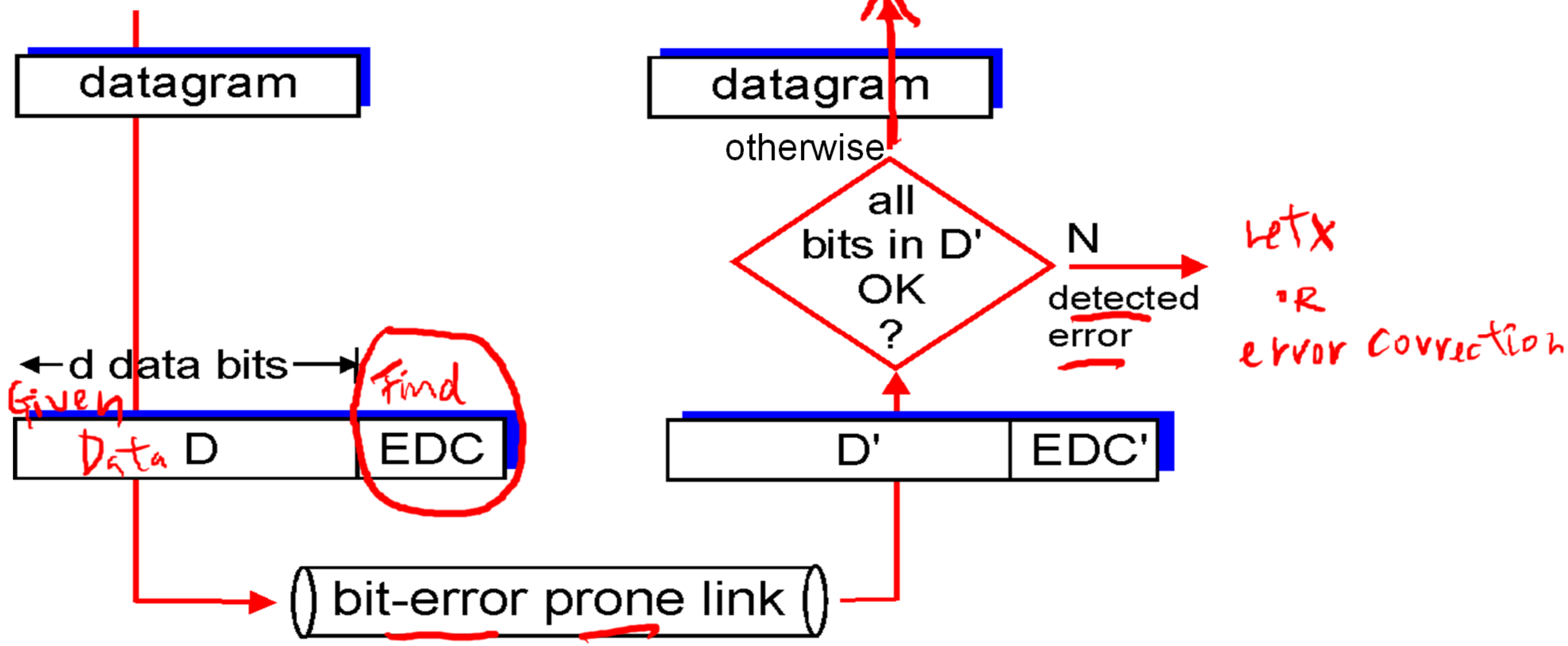
6.6 data center networking

6.7 a day in the life of a web request

Error detection

D = Data protected by error checking, may include header fields
EDC= Error Detection and Correction bits (redundancy)

- Error detection not 100% reliable!
  - protocol may miss some errors, but rarely
  - larger EDC field yields better detection and correction



Parity checking
odd
even

single bit parity:

- detect single bit errors

Given data D

Find EDC

← d data bits → parity bit

0

0111000110101011 | 0

Ass. Odd parity

→ # of "1" is odd (D, EDC)

D: Nine "1" → EDC = 0

⇒ (D, EDC): Nine "1" (odd #)

(D', EDC') = 0011000110101010110

Eight "1" → even → error

two-dimensional bit parity: (ass. Even parity)

- detect and correct single bit errors

D

EDC

row parity

column parity

$d_{1,1}$	. . .	$d_{1,j}$	$d_{1,j+1}$	①
$d_{2,1}$	. . .	$d_{2,j}$	$d_{2,j+1}$	
. . .	. . .	. . .	. . .	
$d_{i,1}$	. . .	$d_{i,j}$	$d_{i,j+1}$	
$d_{i+1,1}$	. . .	$d_{i+1,j}$	$d_{i+1,j+1}$	②

D EDC

10101	1
11110	0
01110	1
00101	0

no errors

D' EDC'

10101	1
10110	0
01110	1
00101	0

parity error

parity error

bit-error prone link

correctable single bit error

Internet checksum (review)

goal: detect "errors" (e.g., flipped bits) in transmitted packet
(note: used at transport layer *only*)

sender:

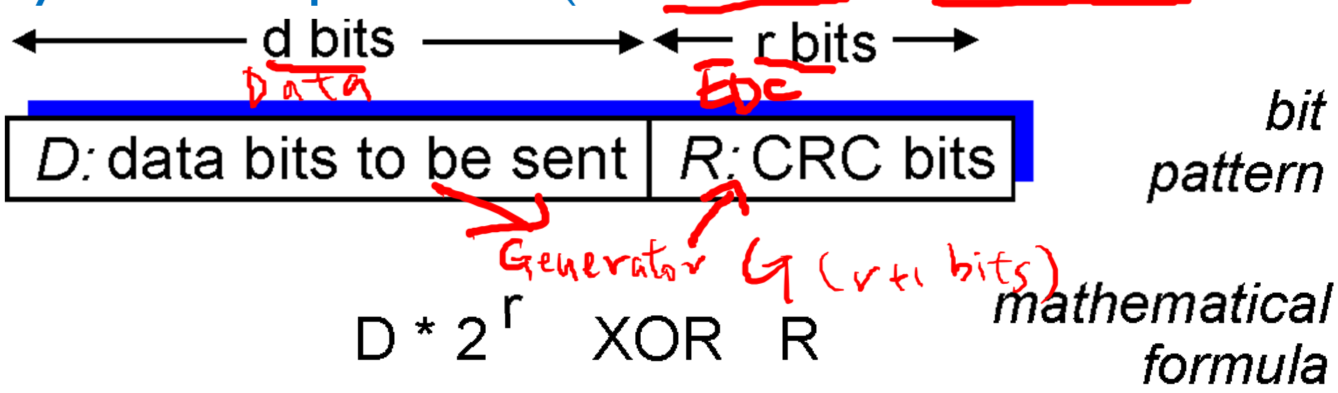
- treat segment contents as sequence of 16-bit integers
- checksum: addition (1's complement sum) of segment contents
- sender puts checksum value into UDP checksum field

receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
  - NO - error detected
  - YES - no error detected. *But maybe errors nonetheless?*

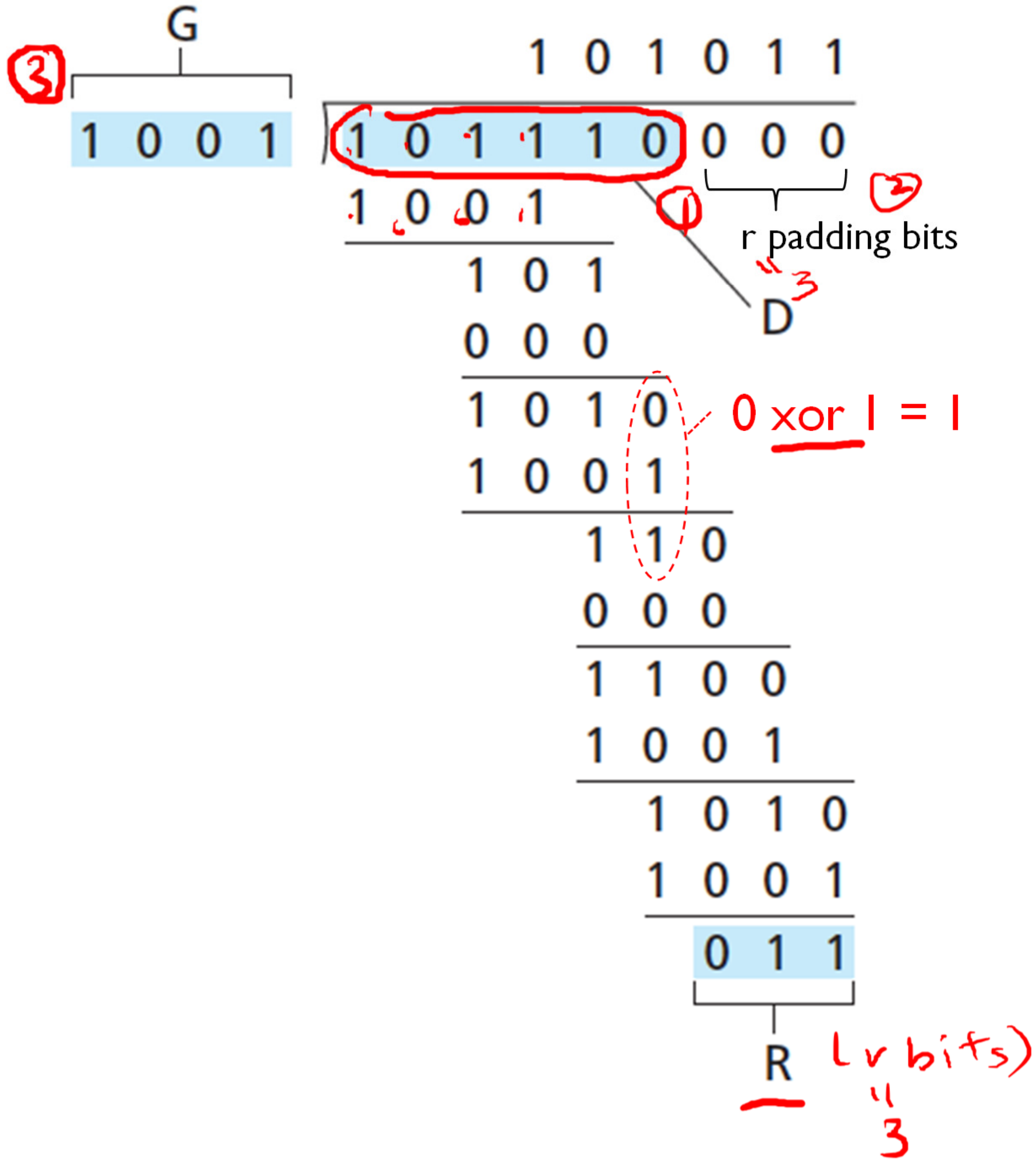
Cyclic redundancy check (CRC)

- more powerful error-detection coding
- view data bits, D, as a binary number
- choose a bit pattern (generator) G with r+1bits
- goal: choose r CRC bits, R, such that
  - <D,R> exactly divisible by G (modulo 2)
  - receiver knows G, divides <D,R> by G. If non-zero remainder: error detected!
  - can detect all burst errors less than r+1 bits (i.e., all consecutive bit errors of r bits or fewer will be detected)
- widely used in practice (Ethernet, 802.11 Wi-Fi, ATM)



CRC example (Encoding)

- CRC Configuration
  - Given G=1001 r+1=4
  - => # of check bits r=3
- Encoding
  - Data D=101110
  - Remainder R= 011 (check bits)
- Transmit
  - {D,R}: $\underbrace{101110}_{D}\underbrace{011}_{R}$



XOR

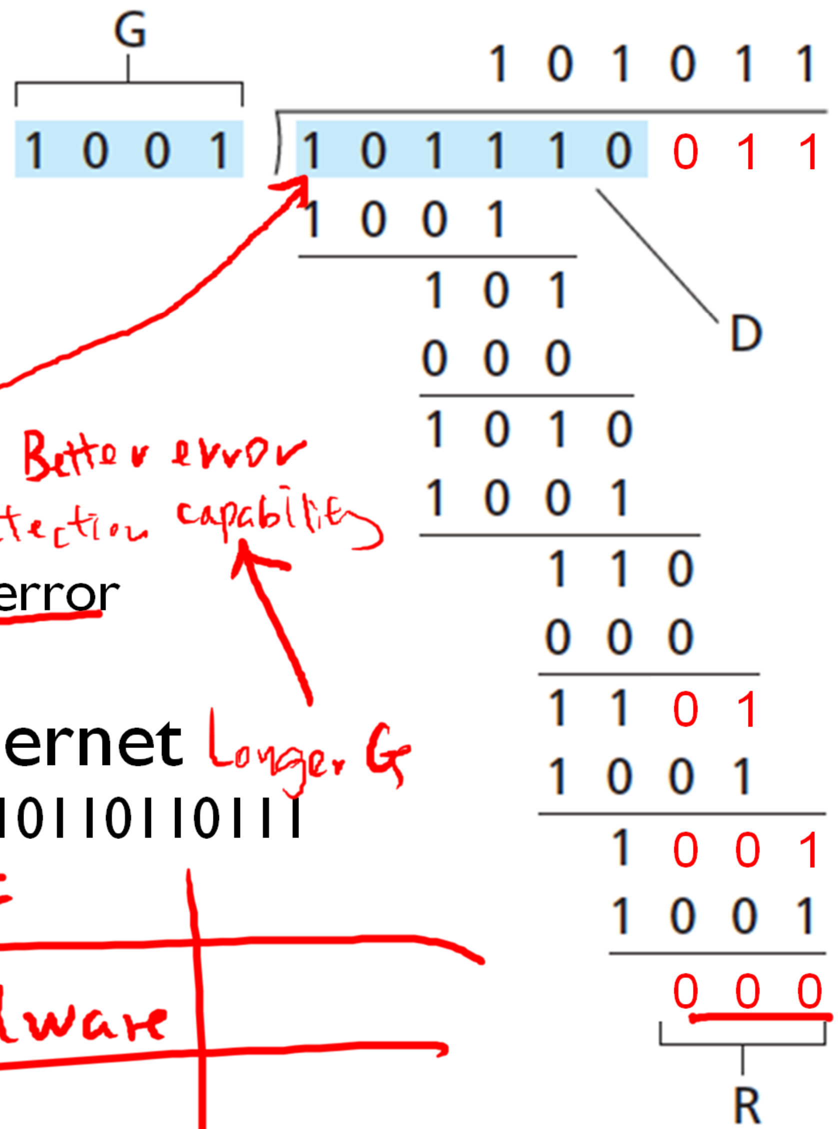
0	0	→ 0
0	1	→ 1
1	0	→ 1
1	1	→ 0

CRC example (Decoding)

- CRC Configuration
  - r=3 bits (# of check bits)
  - G=1001
- Decoding
  - {D,R}=101110011
  - Remainder
    - R= 000 → (very likely) no error
    - R≠ 0 → error detected
- Example: CRC-32 for Ethernet
  - G= 100000100110000010001110110110111

Better error detection capability

Longer G



	Checksum	CRC
Implementation	Software	Hardware
Adv	Simpler	Stronger error detection

Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANS

6.5 link virtualization: MPLS

6.6 data center networking

6.7 a day in the life of a web request

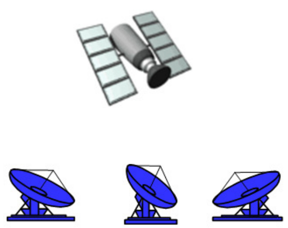
Multiple access links, protocols

two types of "links":

- point-to-point   single sender ⟷ single receiver
  - PPP for dial-up access
  - point-to-point link between Ethernet switch, host
- *broadcast (shared wire or medium)*   multiple senders ⟷ multiple receivers
  - old-fashioned Ethernet
  - upstream HFC
  - 802.11 wireless LAN

shared wire (e.g., cabled Ethernet)

shared RF (e.g., 802.11 Wi-Fi)



shared RF (satellite)

humans at a cocktail party (shared air, acoustical)

Multiple access protocols

- single shared broadcast channel
- two or more simultaneous transmissions by nodes:
  - *collision* if node receives two or more signals at the same time interference

control (MAC)

multiple access protocol

- distributed algorithm that determines how nodes share channel, i.e., determine when node can transmit
- communication about channel sharing must use channel itself!
  - no out-of-band channel for coordination

An "ideal" multiple access protocol

given: broadcast channel of rate R bps

Desirable characteristics:

1. when one node wants to transmit, it can send at rate R.
2. when M nodes want to transmit, each can send at average rate R/M
3. fully decentralized:
   - no special node to coordinate transmissions
   - no synchronization of clocks, slots
4. simple

MAC protocols: taxonomy

three broad classes:

- *channel partitioning*
  - divide channel into smaller "pieces" (time slots, frequency, code)
  - allocate piece to node for exclusive use
- *random access*
  - channel not divided, allow collisions
  - "recover" from collisions
- *"taking turns"*
  - nodes take turns, but nodes with more to send can take longer turns

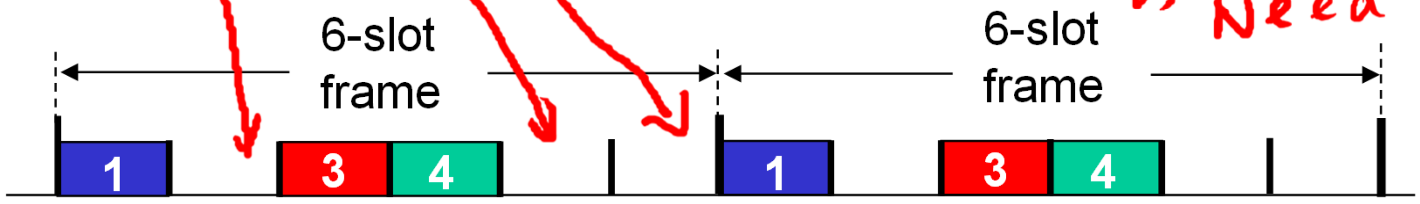
Adv: share channel efficiently & fairly at high load

Channel partitioning MAC protocols: TDMA

TDMA: time division multiple access

- access to channel in "rounds"
- each station gets fixed length slot (length = pkt trans time) in each round
- unused slots go idle
- example: 6-station LAN, 1,3,4 have pkt, slots 2,5,6 idle

Dis:
1) Inefficient at low load
2) Need centralized allocation

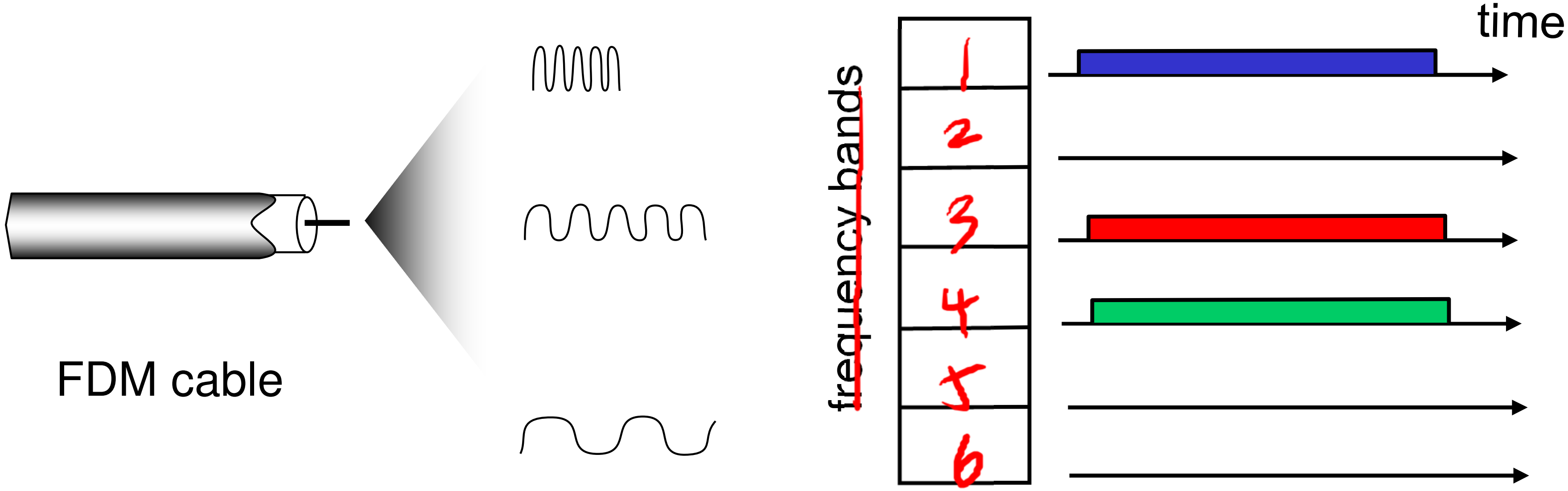


1 round

Channel partitioning MAC protocols: FDMA

FDMA: frequency division multiple access

- channel spectrum divided into frequency bands
- each station assigned fixed frequency band
- unused transmission time in frequency bands go idle
- example: 6-station LAN, 1,3,4 have pkt, frequency bands 2,5,6 idle



Random access protocols

- when node has packet to send
  - transmit at full channel data rate R.
  - no *a priori* coordination among nodes
- two or more transmitting nodes ➜ "collision"
- random access MAC protocol specifies:
  - how to detect collisions
  - how to recover from collisions (e.g., via delayed retransmissions)
- examples of random access MAC protocols:
  - slotted ALOHA
  - ALOHA
  - CSMA, CSMA/CD, CSMA/CA

"Slotted" ALOHA

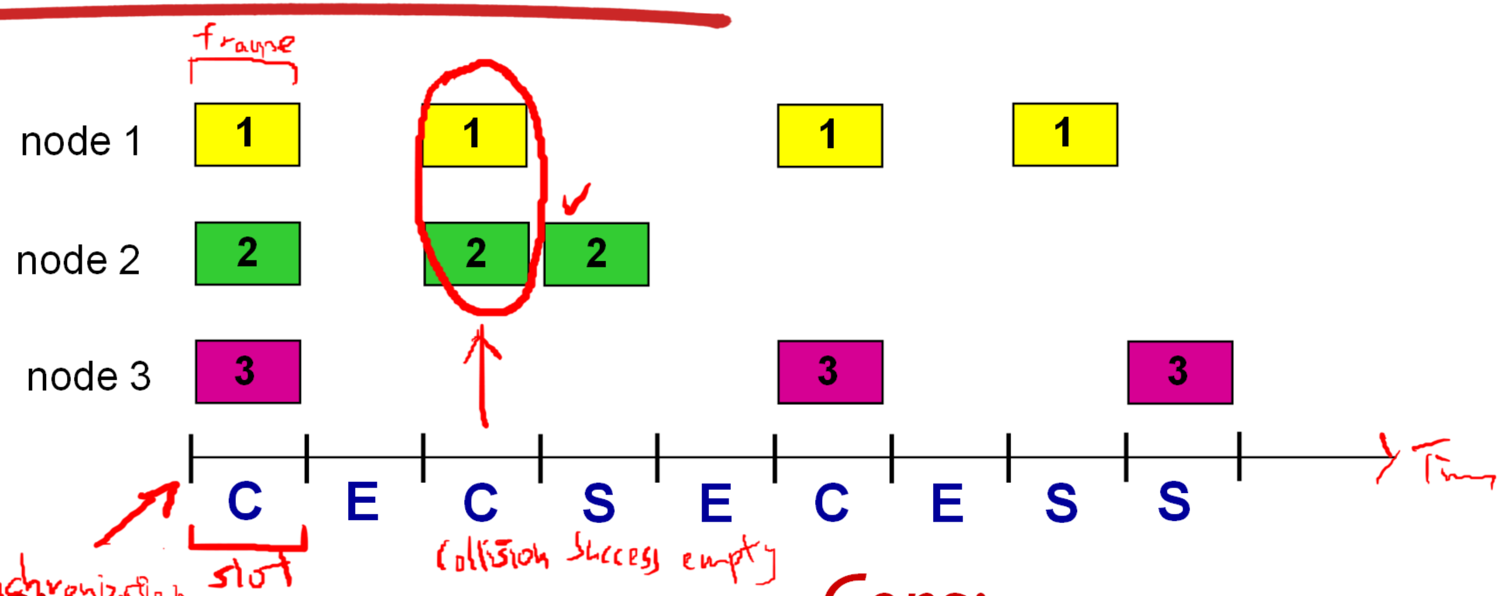
assumptions:

- all frames same size
- time divided into equal size slots (time to transmit 1 frame)
- nodes start to transmit only slot beginning
- nodes are synchronized
- if 2 or more nodes transmit in slot, all nodes detect collision

operation:

- when node obtains fresh frame, transmits in next slot
  - *if no collision*: node can send new frame in next slot
  - *if collision*: node retransmits frame in each subsequent slot with probability p until success

Slotted ALOHA



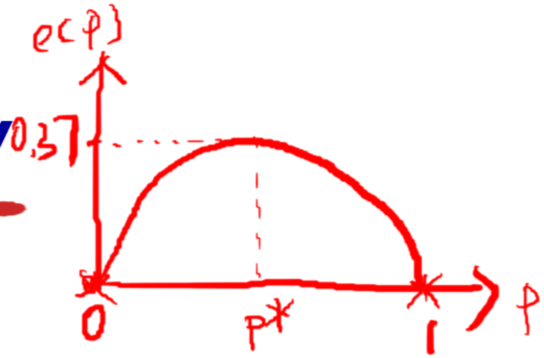
Pros:

- single active node can continuously transmit at full rate of channel
- highly decentralized: only slots in nodes need to be in sync
- simple

Cons:

- collisions, wasting slots
- idle slots
- clock synchronization

Slotted ALOHA: efficiency



efficiency: long-run fraction of successful slots (many nodes, all with many frames to send)

- *suppose: N* nodes with many frames to send, each transmits in slot with probability *p*
- prob that given node has success in a slot success
  only one node tx $= p(1-p)^{N-1}$ other N-1 nodes x tx
- prob that *any* one of the *N* nodes has a success =
  efficiency $e(p) = Np(1-p)^{N-1}$

- max efficiency: find p* that maximizes $Np(1-p)^{N-1}$
- for many nodes, take limit of $Np^*(1-p^*)^{N-1}$ as N goes to infinity, gives:

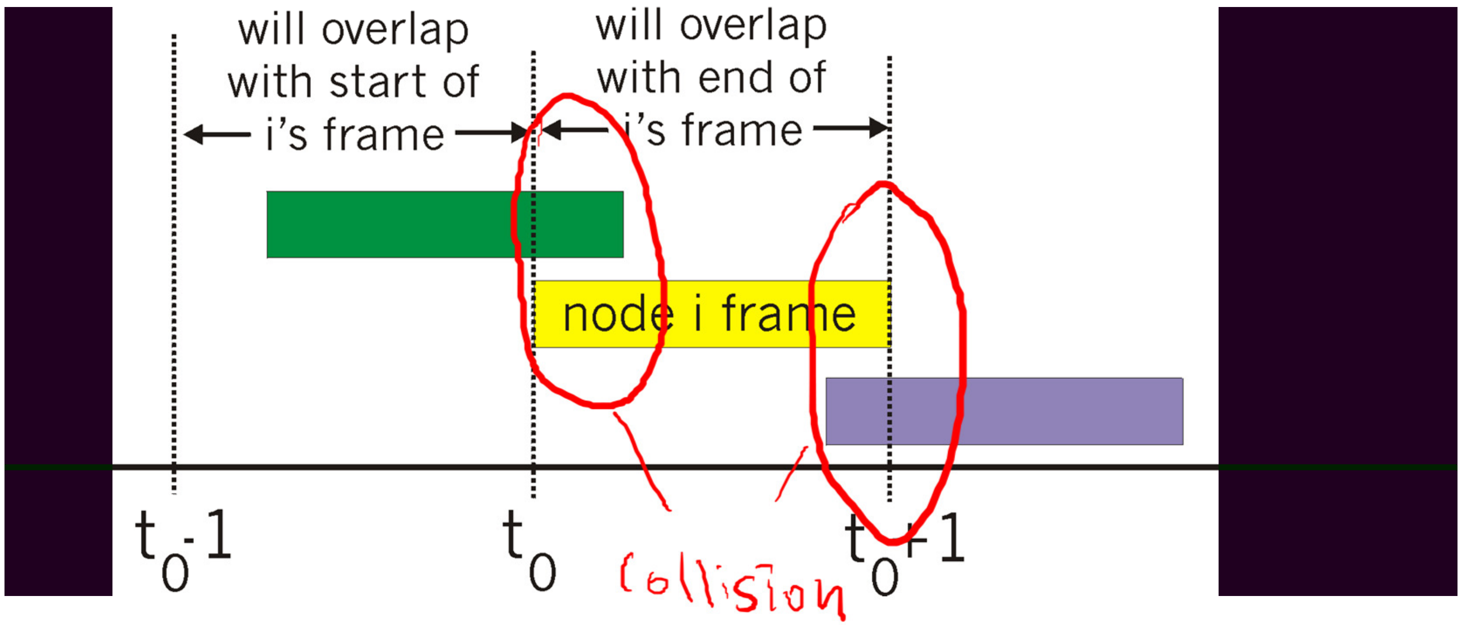
  max efficiency = 1/e = .37

at best: channel used for useful transmissions 37% of time!

!

Pure (unslotted) ALOHA

- unslotted Aloha: simpler, no synchronization
- when frame first arrives
  - transmit immediately
- collision probability increases:
  - frame sent at t_0 collides with other frames sent in $[t_0-1, t_0+1]$



Pure ALOHA efficiency

P(success by given node) = P(node transmits) $\cdot$

P(no other nodes transmit in $[t_0\text{-}1,t_0]$) $\cdot$

P(no other nodes transmit in $[t_0,t_0+1]$)

$$= p \cdot (1-p)^{N-1} \cdot (1-p)^{N-1}$$

$$= p \cdot (1-p)^{2(N-1)}$$

… choosing optimum p and then letting $N \longrightarrow \infty$

max efficiency $= 1/(2e) = .18$ 18% < 37%

(unslotted) (slotted)

even *worse* than slotted Aloha!

CSMA (carrier sense multiple access)

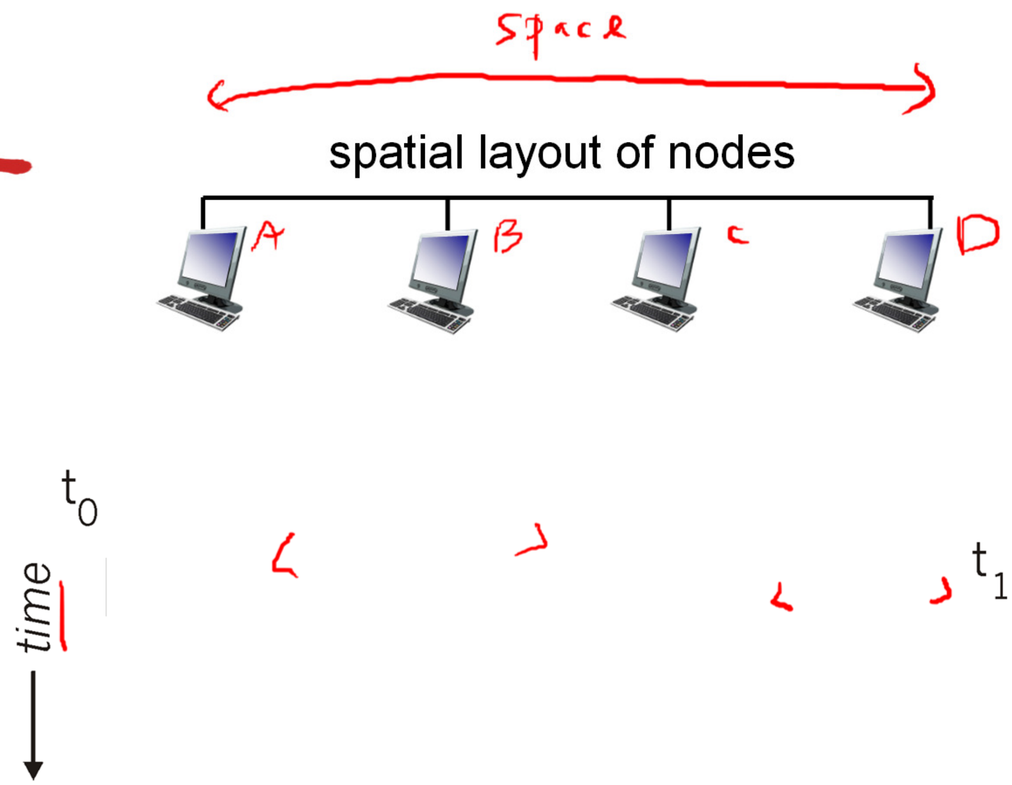
CSMA: listen before transmit:

- if channel sensed idle: transmit entire frame
- if channel sensed busy, defer transmission

- human analogy: don't interrupt others!

Listen before talk

CSMA collisions



- collisions *can* still occur: propagation delay means two nodes may not hear each other's transmission
- Collision -> entire packet transmission time wasted
- distance & propagation delay play role in in determining collision probability

propagation delay ↑

→ ↑ chance of x able to sense → ↑ chance of collision

CSMA/CD (collision detection)

CSMA/CD:

- carrier sensing, deferral as in CSMA
- collisions *detected* within short time

→ colliding transmissions aborted, reducing channel wastage

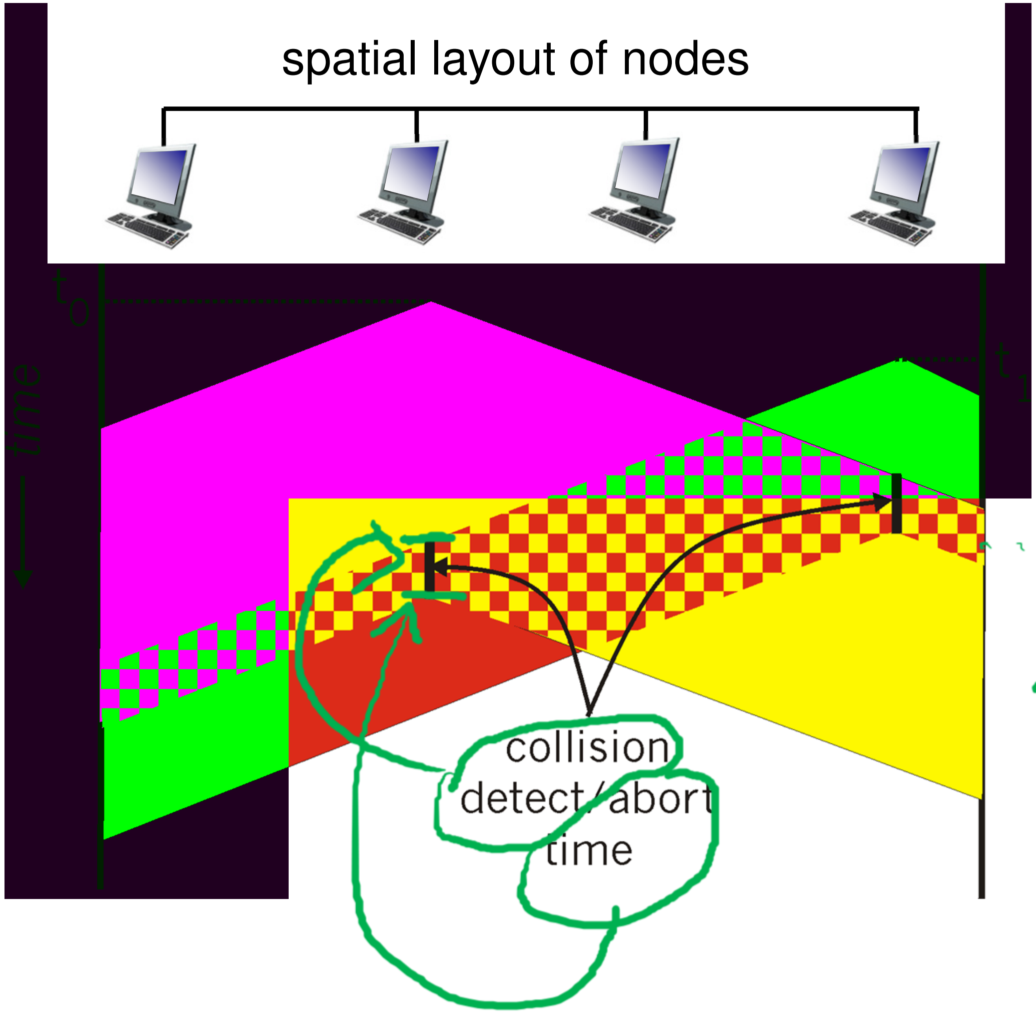
- collision detection in wired vs wireless LANs:
  - easy in wired LANs: measure signal strengths, compare transmitted, received signals
  - difficult in wireless LANs: received signal strength overwhelmed by local transmission strength

hidden terminal → CSMA/CA

- human analogy: the polite conversationalist
  - Listen before speaking.
  - If someone else begins talking at the same time, stop talking.

CSMA

CSMA/CD (collision detection)



overlapping area

→ CH wastage

Ethernet CSMA/CD algorithm

1. NIC receives datagram from network layer, creates frame
2. (a) If NIC senses channel idle, starts frame transmission. CSMA

   (b) If NIC senses channel busy, waits until channel idle, then transmits.
3. While transmitting, NIC monitors for the presence of signal energy coming from other NICs in broadcast channel. CD
4. (a) If NIC transmits entire frame without detecting another transmission, NIC is done with frame! CD

   (b) If NIC detects another transmission while transmitting, aborts transmission.

   Waits a random amount of time (based on *binary exponential backoff algorithm*) and then returns to step 2.

How long should the NIC wait?

retx immediately

wait 1 time slot

m=1, K ∈ {0, 1}

m=2, K ∈ {0, 1, 2, 3}

wait 0 5.12 10.24 15.36 microsec

Binary exponential backoff algorithm:

- after *m*th collision, NIC chooses *K* at random from *{0,1,2, …, 2ᵐ-1}*.
- Ethernet: NIC waits K·512 bit times (i.e., K times the amount of time needed to send 512 bits into the Ethernet) 100 Mbps → 1 bit time = $\frac{1 \text{ bit}}{100\text{Mbps}} = 0.01 \times 10^{-6}$ sec = 0.01 microsec
  - longer backoff interval with more collisions

waiting time
- short → collision again
- long → channel wastage due to long idle time

CSMA/CD efficiency

- t_{prop} = max prop delay between 2 nodes in LAN
- t_{trans} = time to transmit max-size frame

$$efficiency = \frac{1}{1 + 5t_{prop}/t_{trans}}$$

- efficiency goes to 1
  - as t_{prop} goes to 0
  - as t_{trans} goes to infinity
- better performance than ALOHA and simple, cheap, decentralized!

"Taking turns" MAC protocols

channel partitioning MAC protocols:
- share channel *efficiently* and *fairly* at high load
- inefficient at low load: delay in channel access, 1/N bandwidth allocated even if only 1 active node!

random access MAC protocols
- efficient at low load: single node can fully utilize channel
- high load: collision overhead

"taking turns" protocols

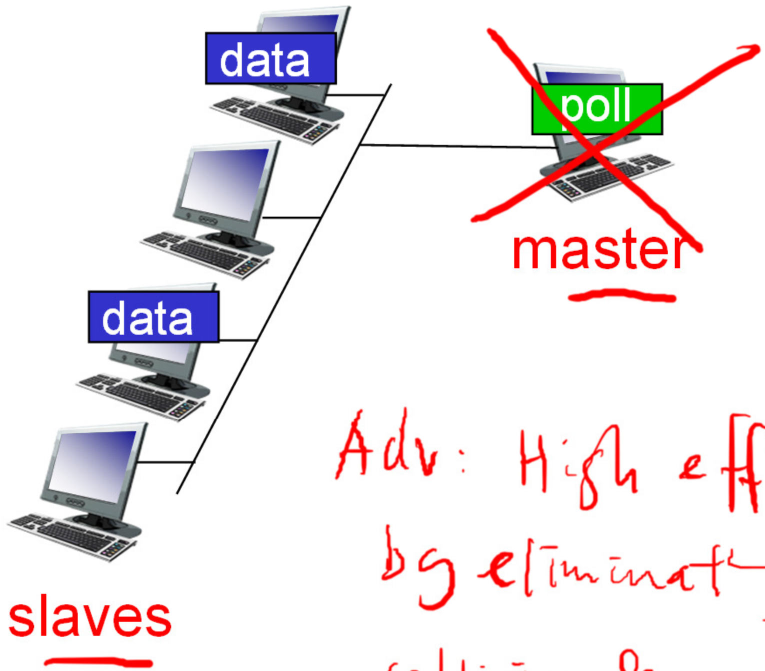
look for best of both worlds!

"Taking turns" MAC protocols

Application: Bluetooth

polling:

- master node "invites" slave nodes to transmit in turn
- typically used with "dumb" slave devices
- concerns:
  - polling overhead
  - latency
  - single point of failure (master)

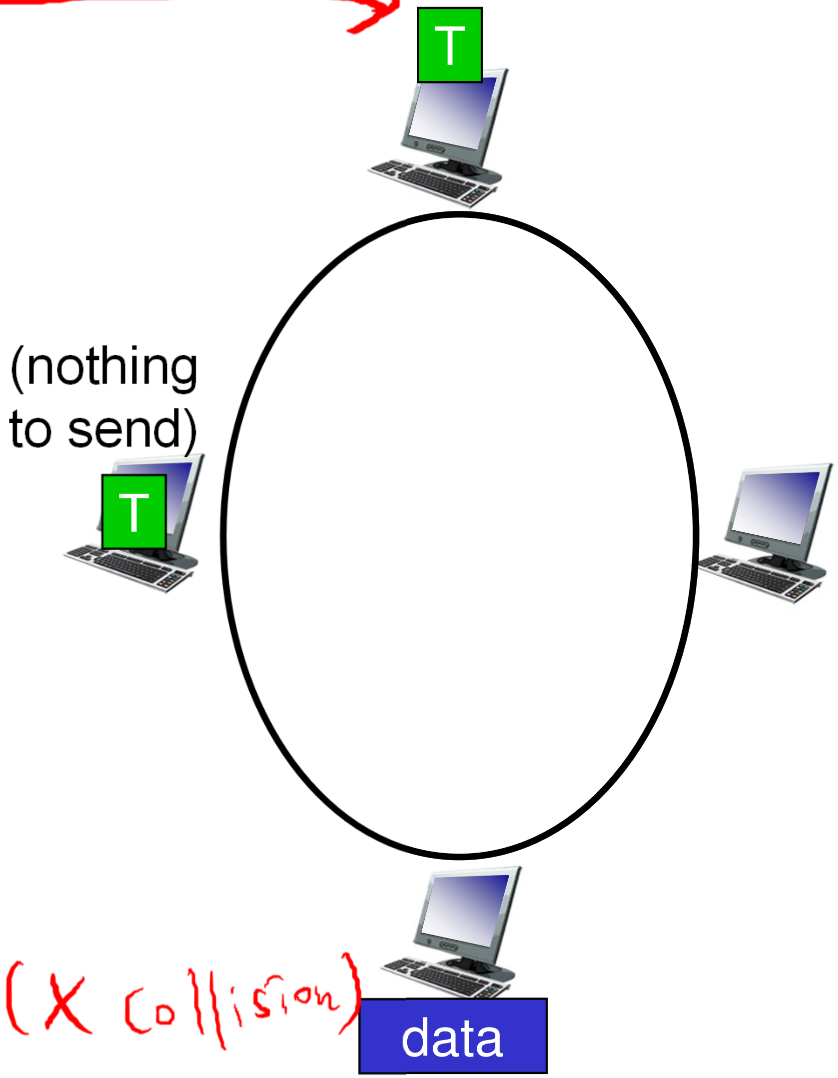


Adv: High efficiency by eliminating collision & empty slots

"Taking turns" MAC protocols

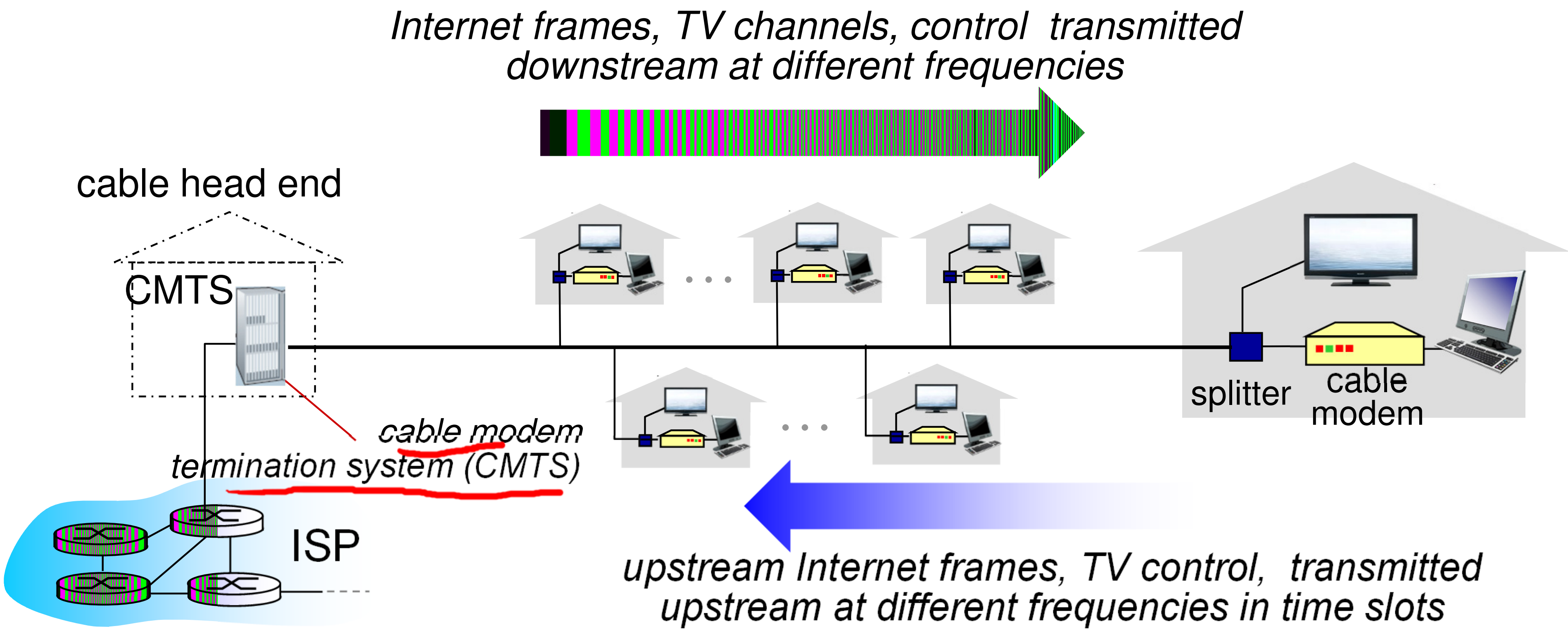
token passing:

- control *token* passed from one node to next sequentially.
- token message
- concerns:
  - token overhead
  - latency
  - single point of failure (token)



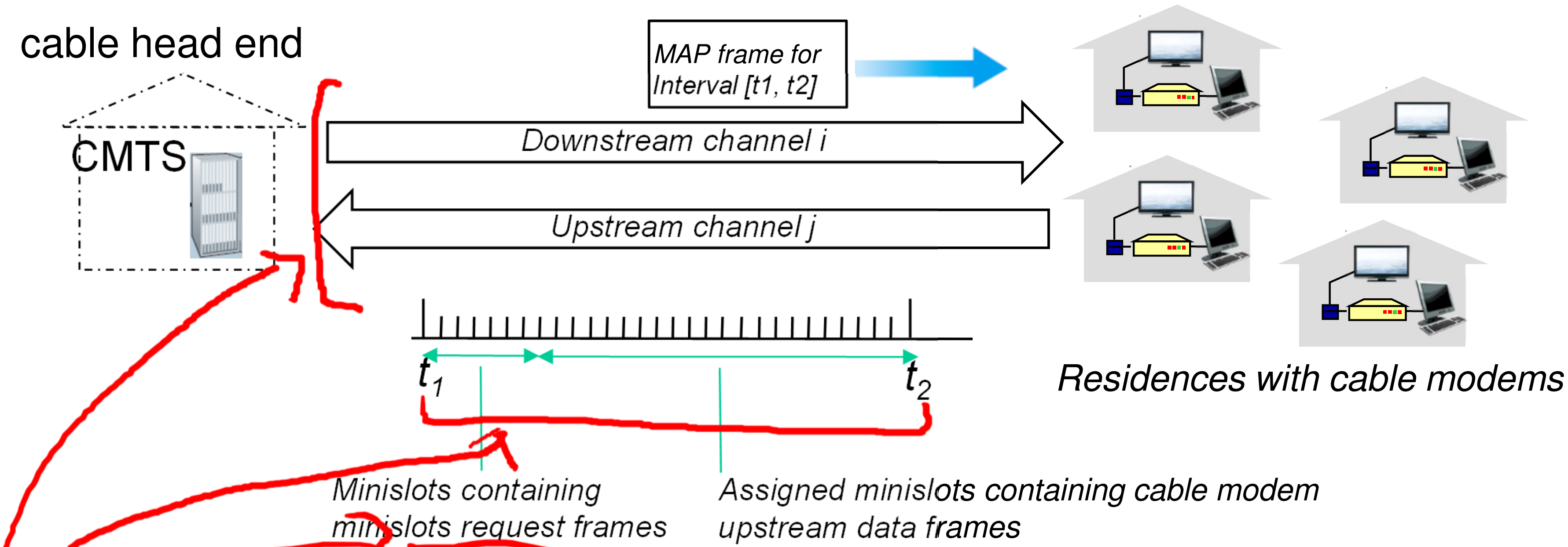
Adv: 1) Decentralized
2) Highly efficient (X collision)

Cable access network: A case study



- *multiple* 40 Mbps downstream (broadcast) channels X collision
  - single CMTS transmits into channels
- *multiple* 30 Mbps upstream channels
  - *multiple access: all* users contend for certain upstream channel time slots (others assigned)

Cable access network



Data-Over-Cable Service Interface Specifications (DOCSIS): data-over-cable service interface spec

- FDM over upstream, downstream frequency channels
- TDM upstream: some slots assigned, some have contention
  - downstream MAP frame: assigns upstream slots
  - request for upstream slots (and data) transmitted random access (binary backoff) in selected slots

Summary of MAC protocols

- *channel partitioning,* by time, frequency or code
  - Time Division, Frequency Division
- *random access* (dynamic),
  - ALOHA, S-ALOHA, CSMA, CSMA/CD
  - CSMA/CD used in Ethernet
  - CSMA/CA used in Wi-Fi (IEEE 802.11)
- *taking turns*
  - polling from central site, token passing
  - Bluetooth, fiber distributed data interface (FDDI), token ring

Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs
- addressing, ARP
- Ethernet
- switches
- VLANS

6.5 link virtualization: MPLS

6.6 data center networking

6.7 a day in the life of a web request

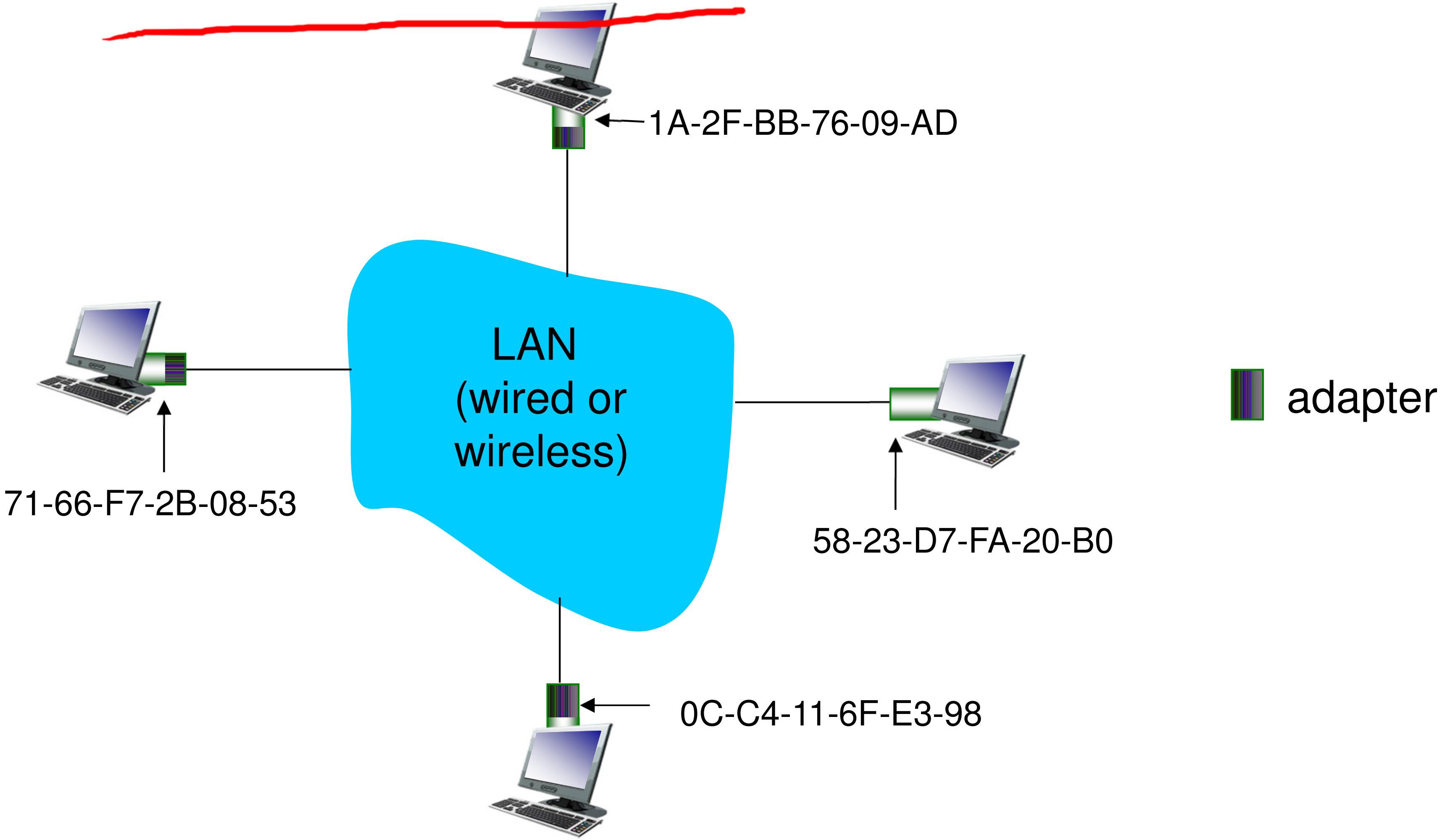
MAC addresses

- 32-bit IP address:
  - *network-layer* address for interface
  - used for layer 3 (network layer) forwarding
- MAC (or LAN or physical or Ethernet) address:
  - function: *used 'locally" to get frame from one interface to another physically-connected interface (same network, in IP-addressing sense)*
  - 48 bit MAC address (for most LANs) burned in NIC ROM, also sometimes software settable
  - e.g.: 1A-2F-BB-76-09-AD

hexadecimal (base 16) notation
(each "number" represents 4 bits)

MAC addresses

each adapter (i.e., NIC) on LAN has unique *MAC* address (i.e., no two adapters have the same address)



MAC addresses (more)

- MAC address allocation administered by IEEE
- manufacturer buys portion of MAC address space (to assure uniqueness)
- MAC flat address ➜ portability
  - can move LAN card from one LAN to another

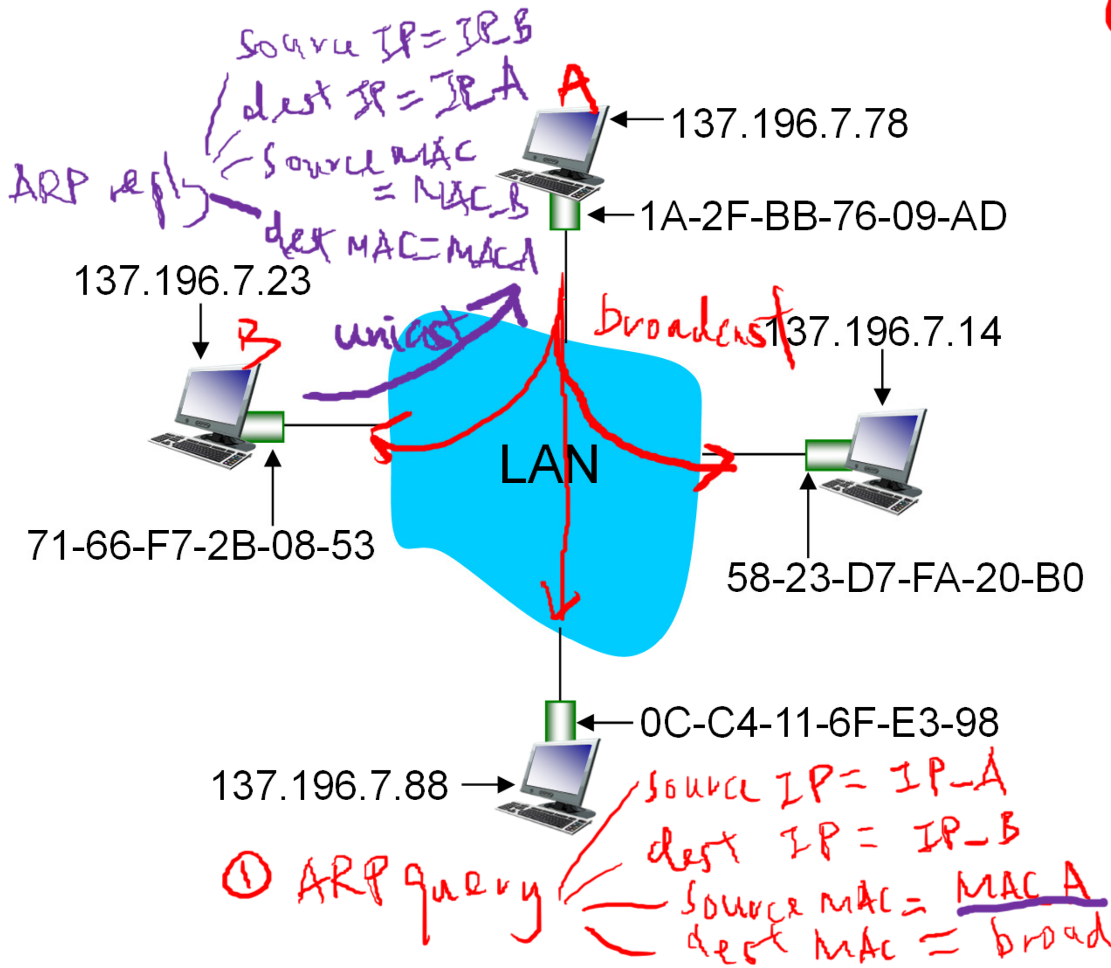
change?

- IP hierarchical address *not* portable
  - address depends on IP subnet to which node is attached

ARP: address resolution protocol

same LAN

Question: how to determine interface's MAC address, knowing its IP address?



- A wants to send datagram to B Ass: A ✓ know B's IP addr; × .. B's MAC addr
  - Problem: B's MAC address not in A's ARP table.
- ① A broadcasts ARP query packet, containing B's IP address
  - dest MAC address = FF-FF-FF-FF-FF-FF (broadcast address)
  - all nodes on LAN receive ARP query
- ② B receives ARP packet, replies to A with its (B's) MAC address
  - frame sent to A's MAC address (unicast)

ARP protocol: same LAN

ARP table: each IP node (host, router) on LAN has table

- IP/MAC address mappings for some LAN nodes:

  < IP address; MAC address; TTL>

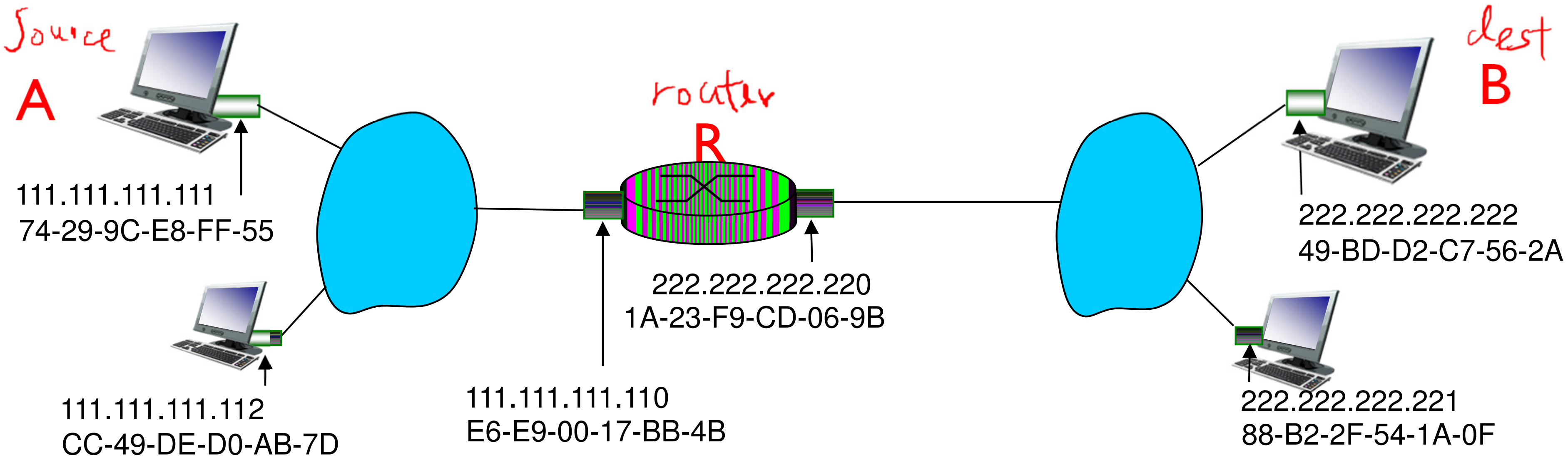
- TTL (Time To Live): time after which address mapping will be forgotten (typically 20 min)

- A caches (saves) IP-to-MAC address pair in its ARP table until information becomes old (times out)
  - soft state: information that times out (goes away) unless refreshed
- ARP is "plug-and-play":
  - nodes create their ARP tables *without intervention from net administrator*

Addressing: routing to another LAN

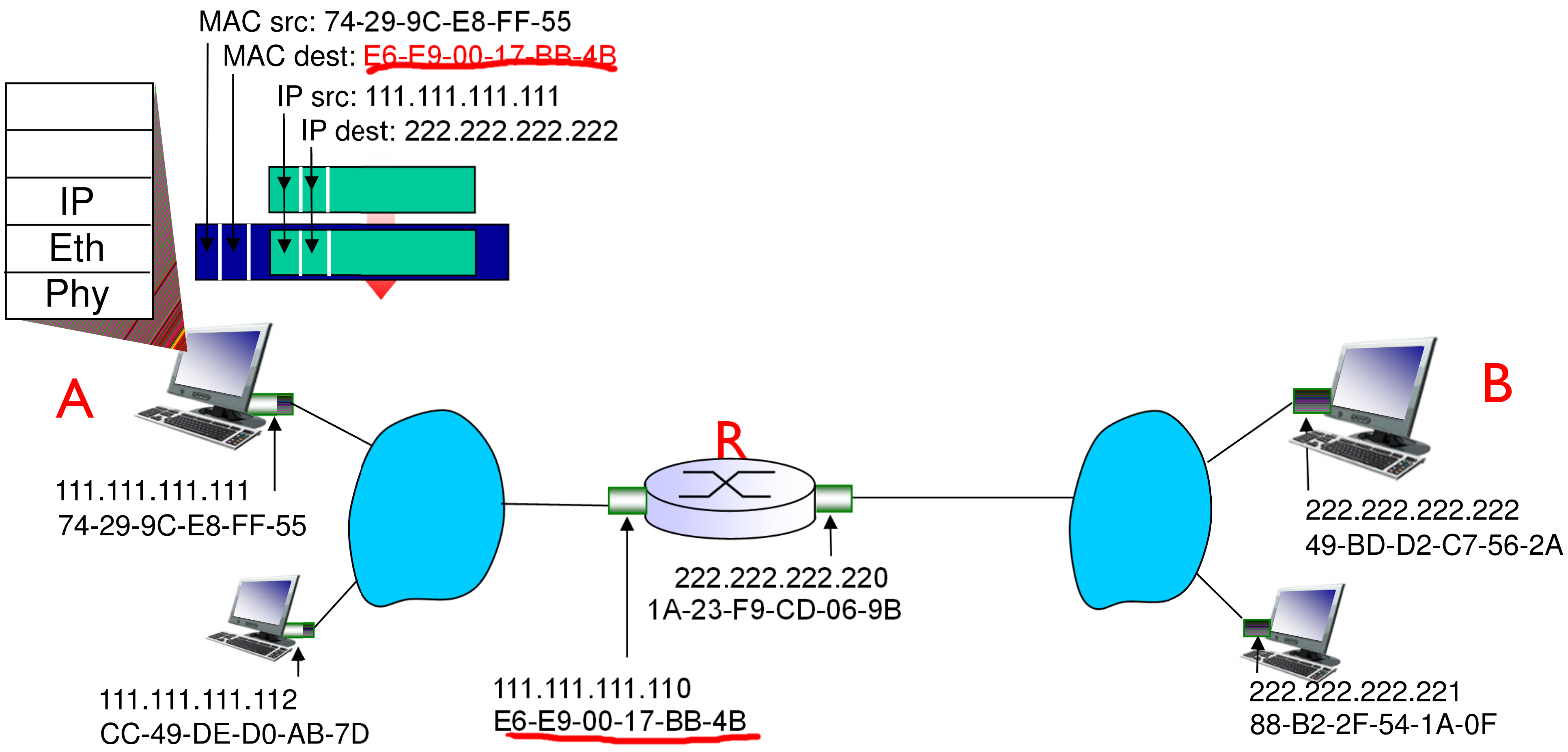
walkthrough: send datagram from A to B via R

- focus on addressing – at IP (datagram) and MAC layer (frame)
- assume A knows B' s IP address
- assume A knows IP address of first hop router, R (how?) DHCP
- assume A knows R' s MAC address (how?) ARP



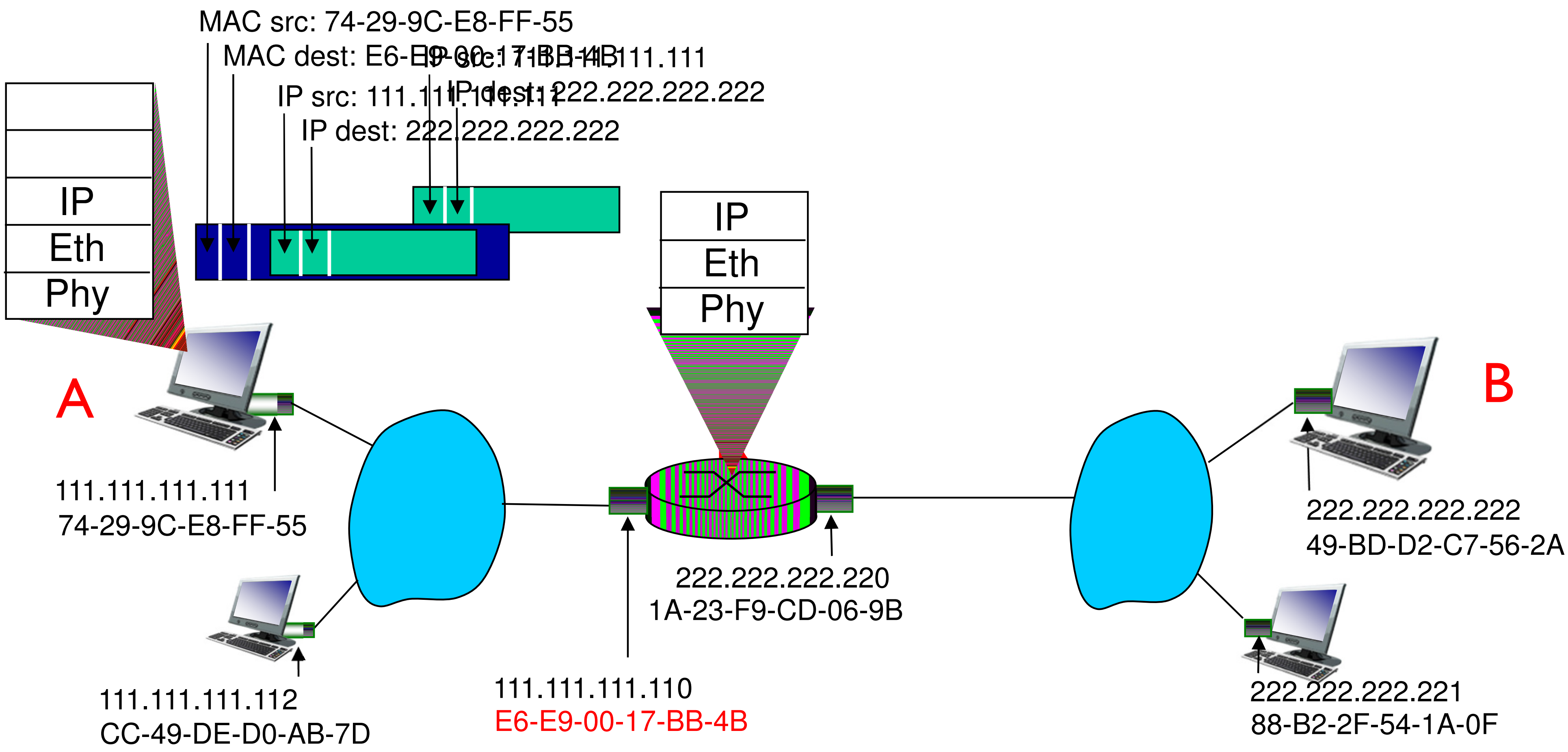
Addressing: routing to another LAN

- A creates IP datagram with IP source A, destination B
- A creates link-layer frame with R's MAC address as dest, frame contains A-to-B IP datagram



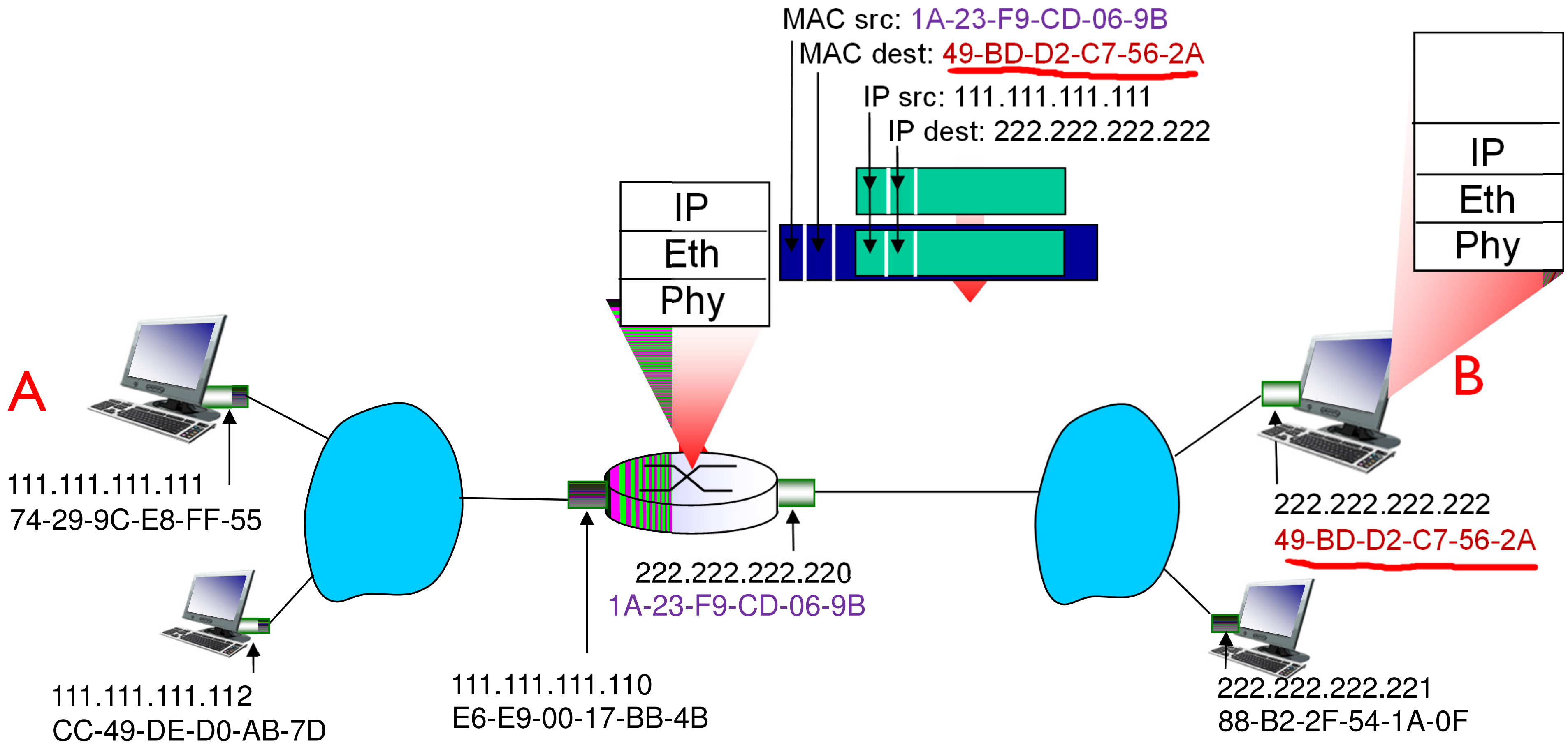
Addressing: routing to another LAN

- frame sent from A to R
- frame received at R, datagram removed, passed up to IP



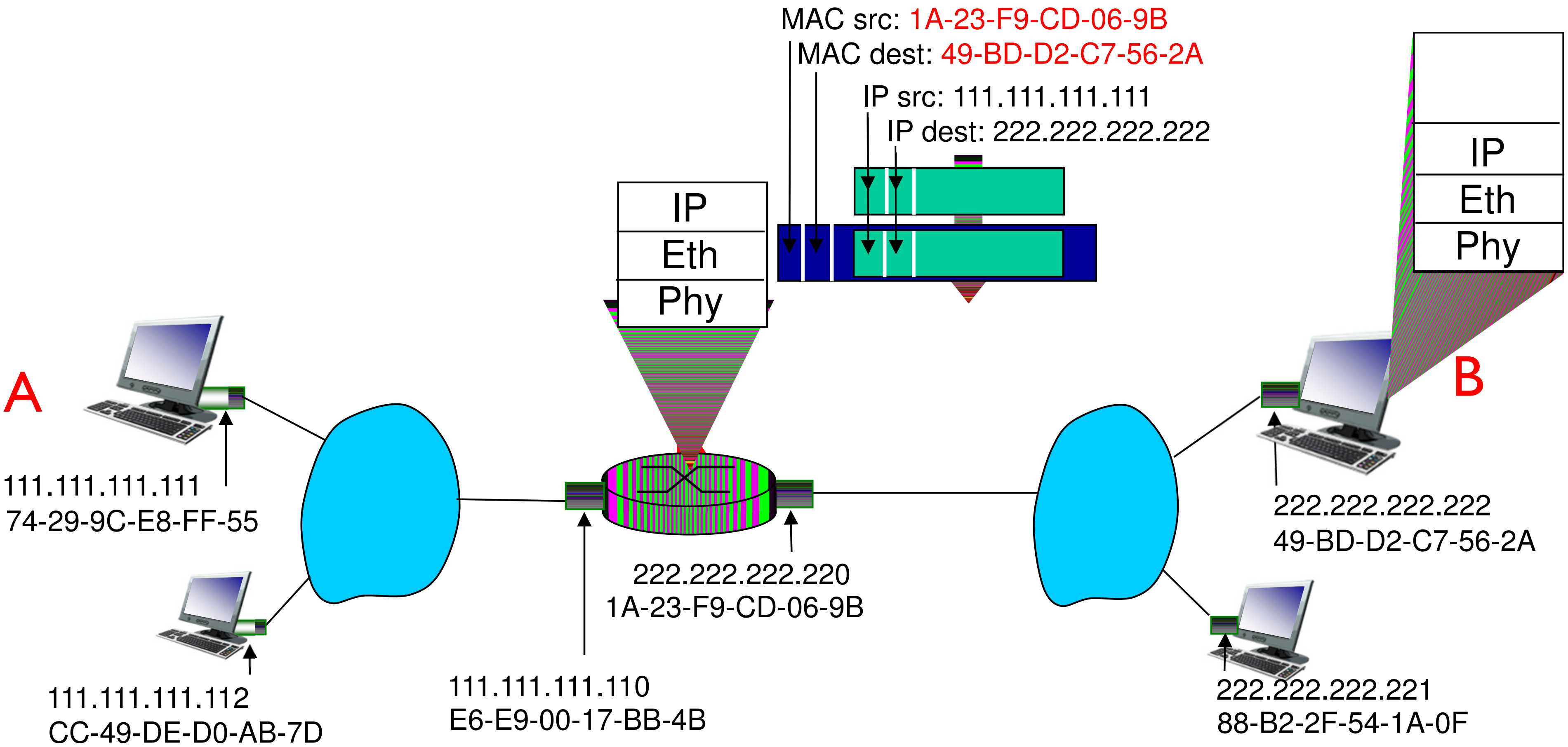
Addressing: routing to another LAN

- R forwards datagram with IP source A, destination B
- R creates link-layer frame with B's MAC address as dest, frame contains A-to-B IP datagram



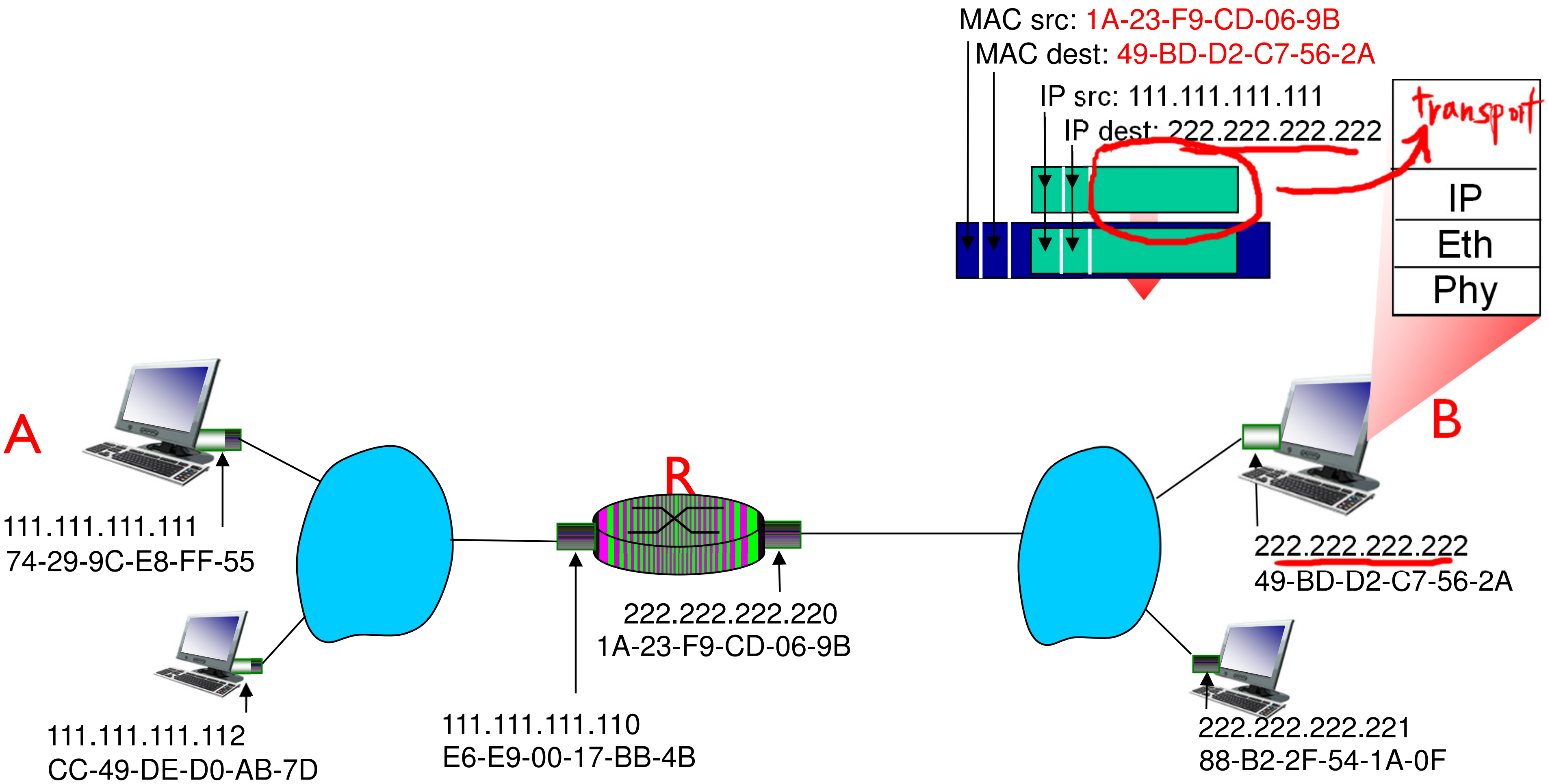
Addressing: routing to another LAN

- R forwards datagram with IP source A, destination B
- R creates link-layer frame with B's MAC address as dest, frame contains A-to-B IP datagram



Addressing: routing to another LAN

- R forwards datagram with IP source A, destination B
- R creates link-layer frame with B's MAC address as dest, frame contains A-to-B IP datagram



Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs
- addressing, ARP
- Ethernet
- switches
- VLANS

6.5 link virtualization: MPLS

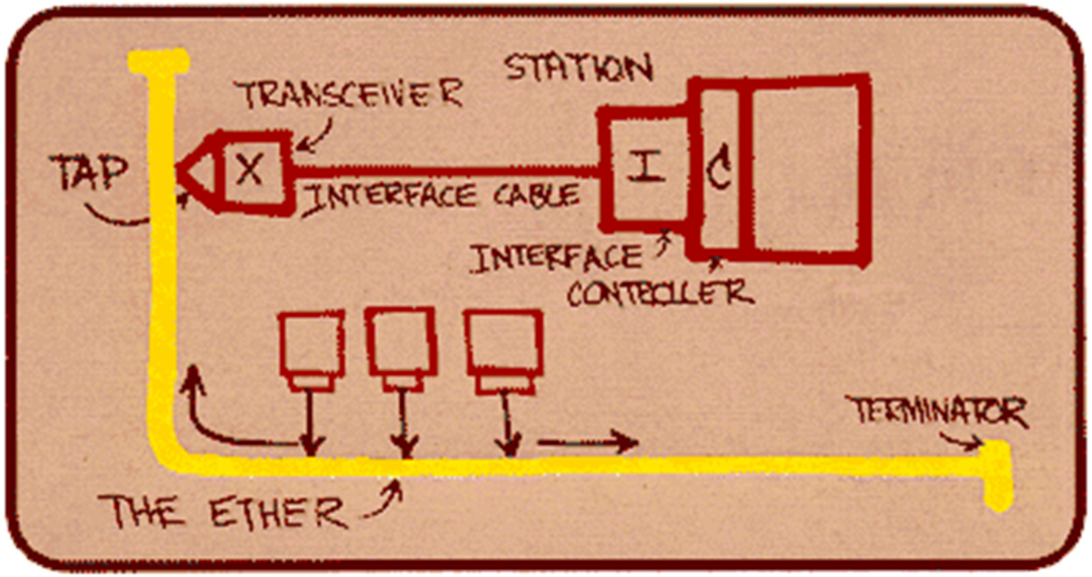
6.6 data center networking

6.7 a day in the life of a web request

Ethernet

"dominant" wired LAN technology:

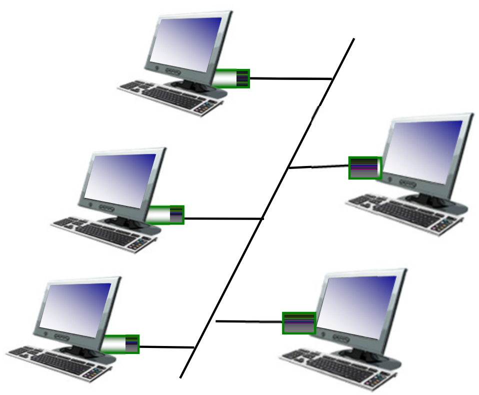
- cheap $20 for NIC
- first widely used LAN technology
- simpler, cheaper than token LANs and ATM
- kept up with speed race: 10 Mbps – 10 Gbps



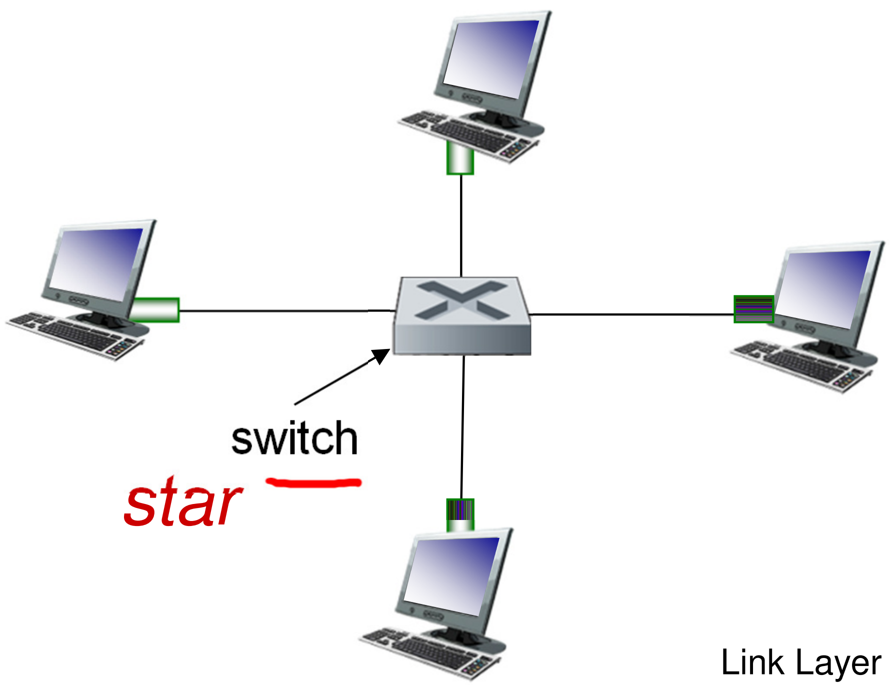
Metcalfe's Ethernet sketch

Ethernet: physical topology

- *bus:* popular through mid 90s
  - all nodes in same collision domain (can collide with each other)
- *star:* prevails today
  - active *switch* in center
  - nodes do not collide with each other



bus: coaxial cable



star

Ethernet frame structure

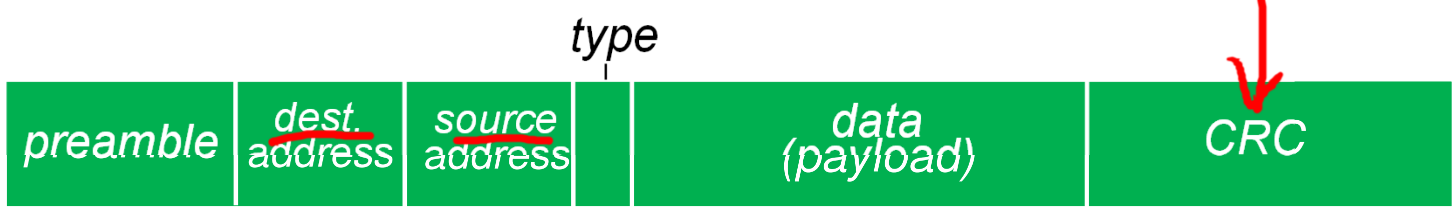
sending adapter encapsulates IP datagram (or other network layer protocol packet) in Ethernet frame

preamble:

- 8 bytes:
  - 7 bytes with pattern 10101010
  - followed by one byte with pattern 10101011
- used to synchronize receiver and sender clock rates

Ethernet frame structure (more)

- *addresses:* 6 byte source, destination MAC addresses
  - if adapter receives frame with matching destination address, or with broadcast address (e.g. ARP packet), it passes data in frame to network layer protocol
  - otherwise, adapter discards frame
- *type:* indicates higher layer protocol (mostly IP but others possible, e.g., Novell IPX, AppleTalk)
- *CRC:* cyclic redundancy check at receiver
  - error detected: frame is dropped



Ethernet: unreliable, connectionless

Adv: 1) Simple & cheap

- *connectionless:* no handshaking between sending and receiving NICs

Dis: 1)

- *unreliable:* receiving NIC doesn't send acks or nacks to sending NIC => Gap in Seq # under UDP
  - data in dropped frames recovered only if initial sender uses higher layer rdt (e.g., TCP), otherwise dropped data lost
- Ethernet's MAC protocol: *unslotted CSMA/CD with binary backoff*

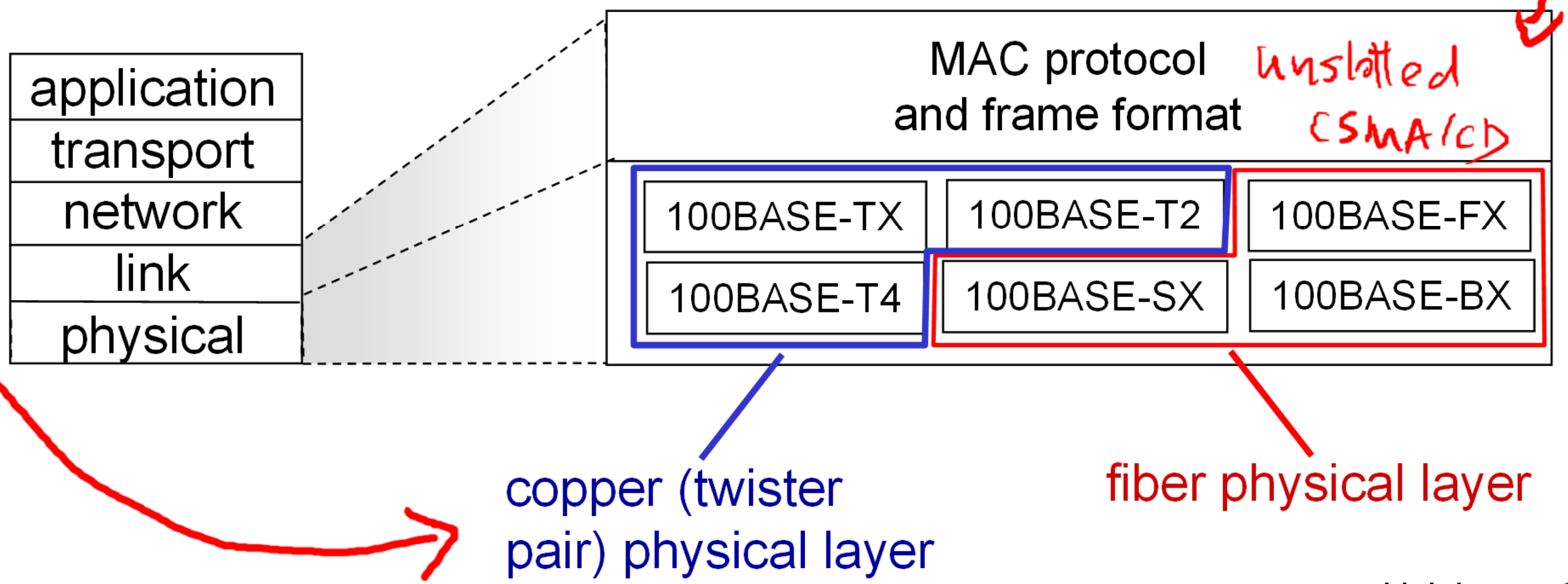
S → r → CRC check?

× ACK.. pass → data to upper layer

× NAK .. fail → discard frame

802.3 Ethernet standards: link & physical layers

- *many* different Ethernet standards
  - common MAC protocol and frame format
  - different speeds: 2 Mbps, 10 Mbps, 100 Mbps, 1Gbps, 10G bps
  - different physical layer media: fiber, cable



Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs
- addressing, ARP
- Ethernet
- switches
- VLANS

6.5 link virtualization: MPLS

6.6 data center networking

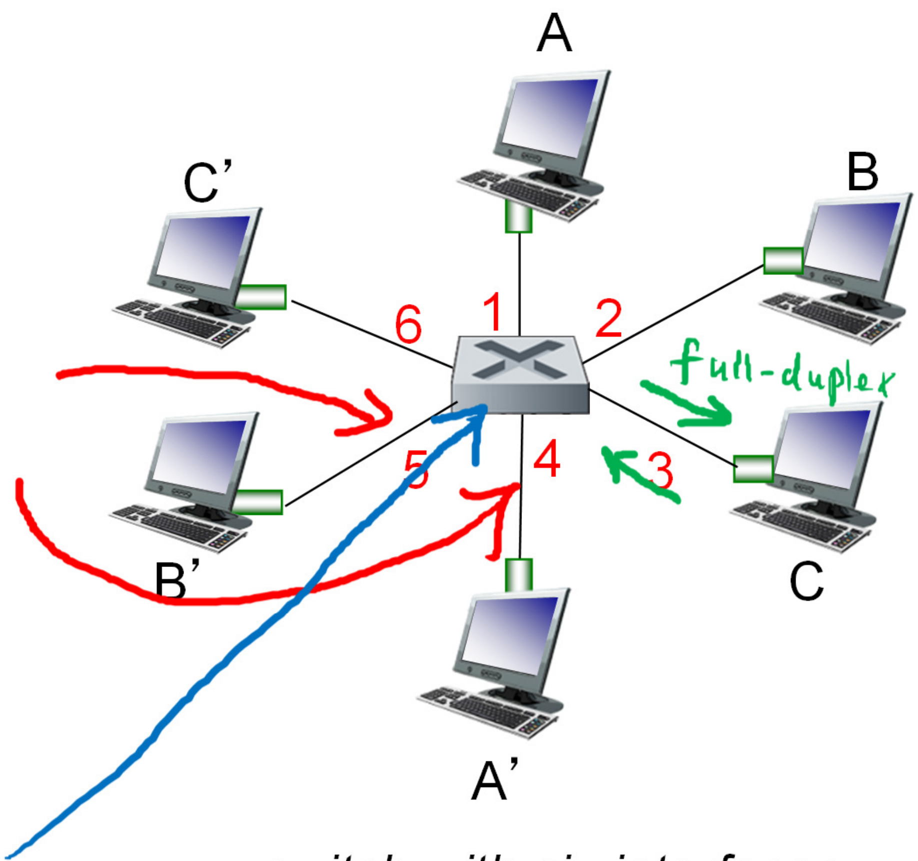
6.7 a day in the life of a web request

Ethernet switch

- link-layer device: takes an *active* role
  - store, forward Ethernet frames
  - examine incoming frame's MAC address, selectively forward frame to one-or-more outgoing links when frame is to be forwarded on segment, uses CSMA/CD to access segment
- *transparent* × need MAC addr
  - hosts are unaware of presence of switches
- *plug-and-play, self-learning*
  - switches do not need to be configured

Switch: *multiple* simultaneous transmissions

- hosts have dedicated, direct connection to switch
- switches buffer packets
- Ethernet protocol used on *each* incoming link, but no collisions
  - each link is its own collision domain
- *Full duplex:* Any switch interface can send & receive at the same time
- *switching:* A-to-A' and B-to-B' can transmit simultaneously, without collisions

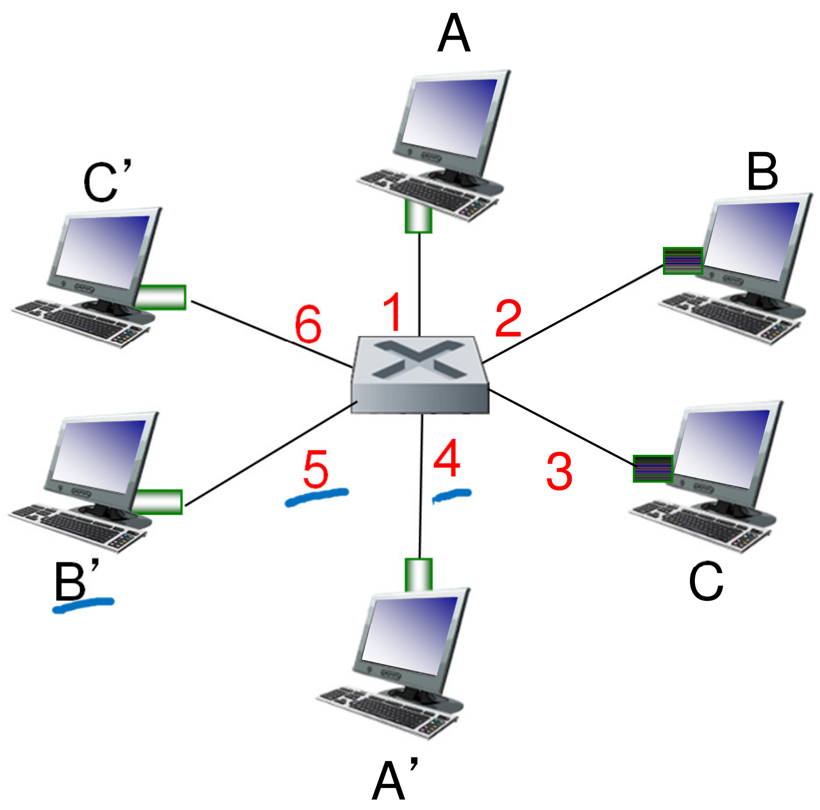


switch with six interfaces (1,2,3,4,5,6)

Switch forwarding table

Q: how does switch know A' reachable via interface 4, B' reachable via interface 5?

- *A: each switch has a switch table, each entry:*
  - *(MAC address of host, interface to reach host, time stamp)*
  - *looks like a routing table!*



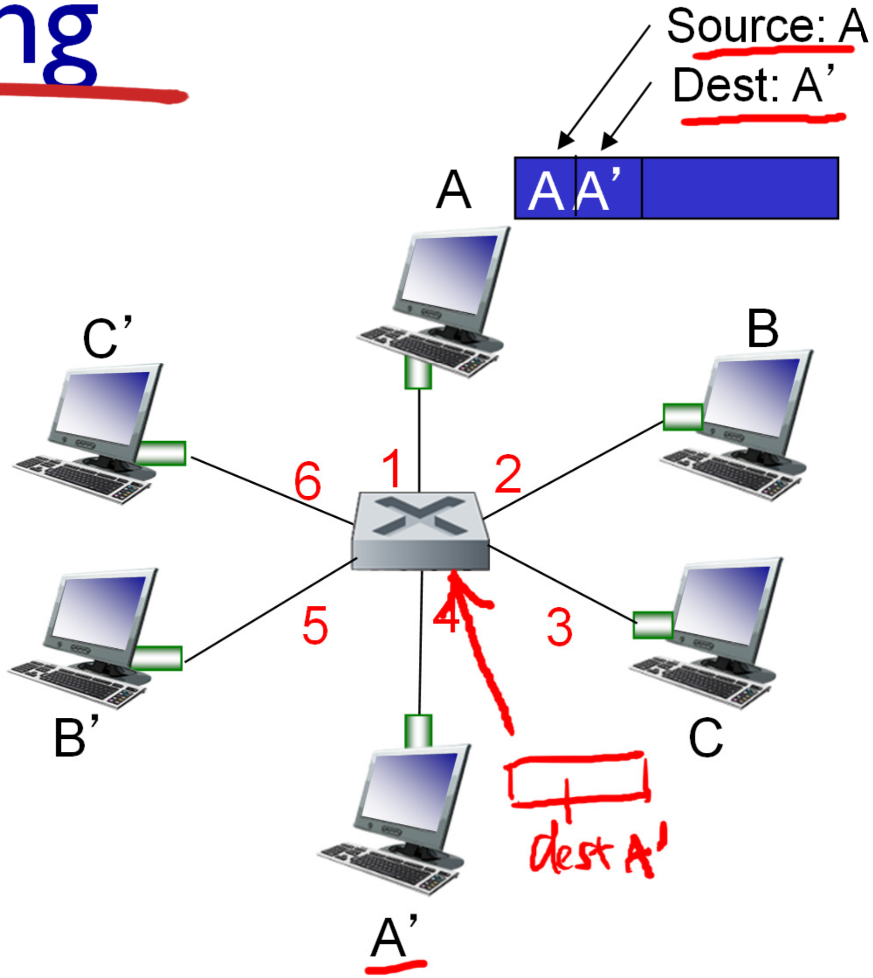
switch with six interfaces (1,2,3,4,5,6)

Q: how are entries created, maintained in switch table?

- *something like a routing protocol?*

Switch: "self-learning"

- switch *learns* which hosts can be reached through which interfaces
  - when frame received, switch "learns" location of sender: incoming LAN segment
  - records sender/location pair in switch table

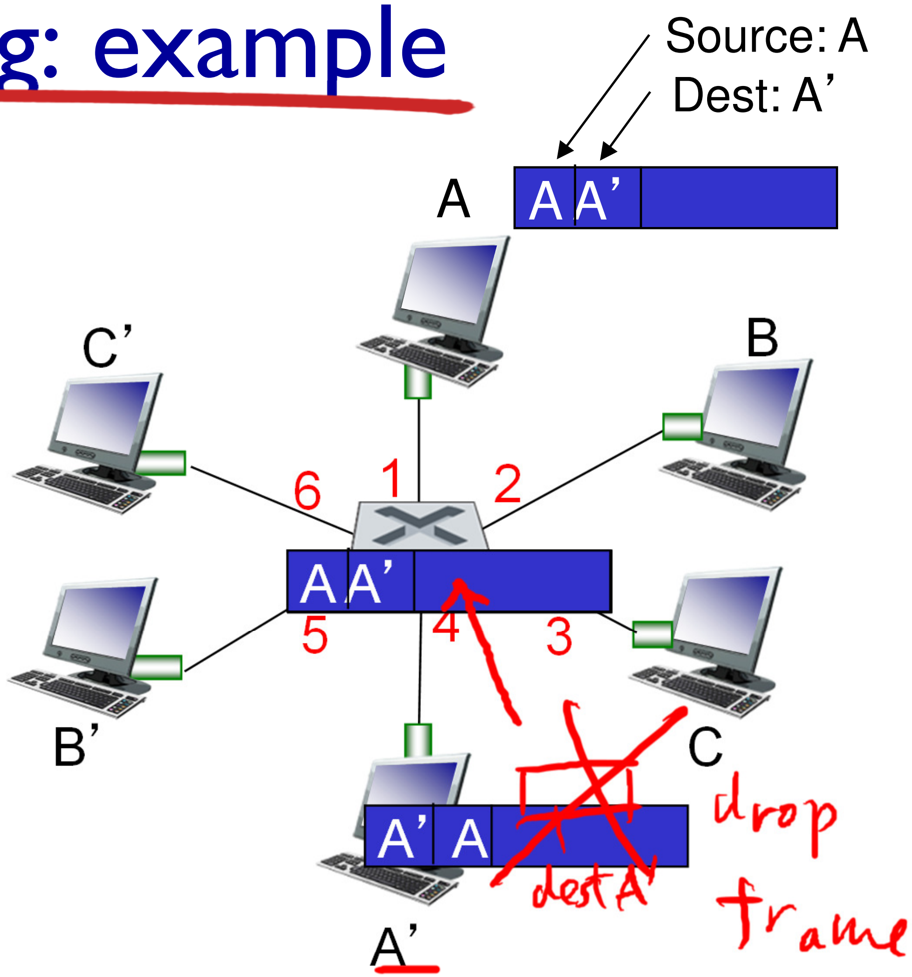


MAC addr	interface	TTL
A	1	60

Switch table (initially empty)

Self-learning, forwarding: example

- frame destination, A', location unknown: *flood*
- destination A location known: *selectively send on just one link*



MAC addr	interface	TTL
A	*1*	*60*
A'	*4*	*60*

switch table (initially empty)

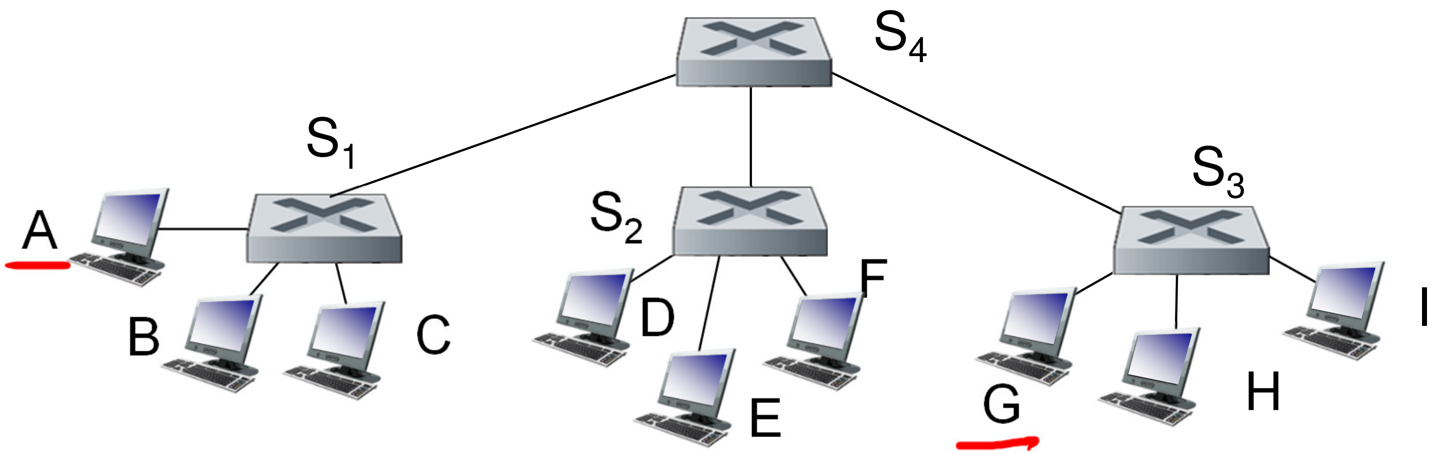
Switch: frame filtering/forwarding

when frame received at switch:

1. record incoming link, MAC address of sending host
2. index switch table using MAC destination address
3. if entry found for destination
   then {
   if destination on segment from which frame arrived
   then drop frame
   else forward frame on interface indicated by entry
   "selectively send"
   }
   else flood /* forward on all interfaces except arriving interface */

Interconnecting switches

- switches can be connected together



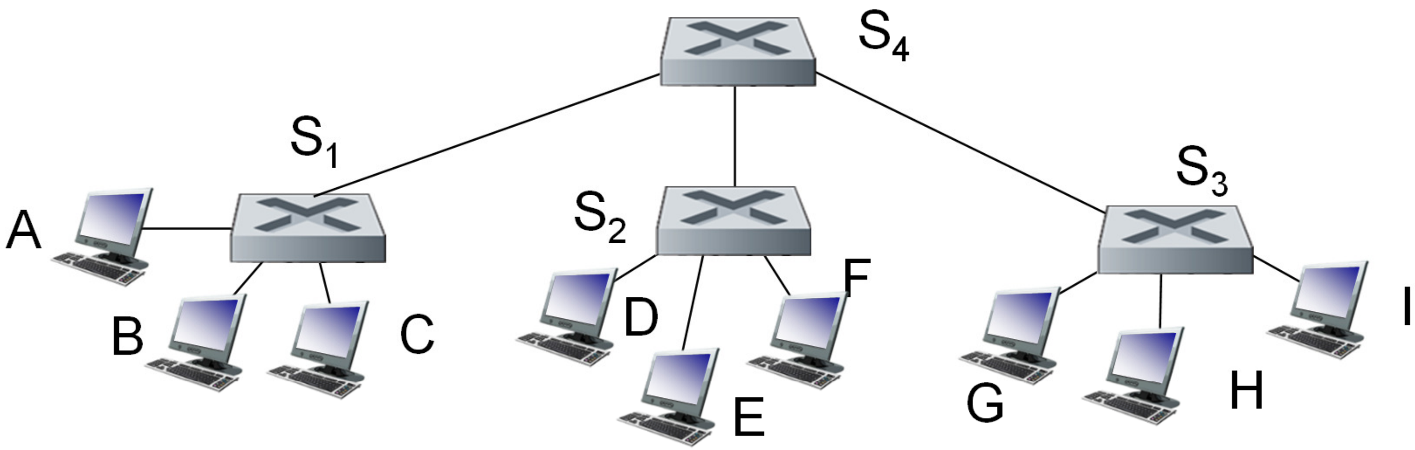
Q: -sending from A to G

-how does S_1 know to forward frame destined to G via S_4 and S_3?

- *A:* self-learning! (works *exactly* the same as in single-switch case!)

Self-learning multi-switch example

Suppose A sends frame to G, G responds to A



- *Q:* show switch tables and packet forwarding in S_1, S_2, S_3, S_4

Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs
- addressing, ARP
- Ethernet
- switches
- VLANS

6.5 link virtualization: MPLS

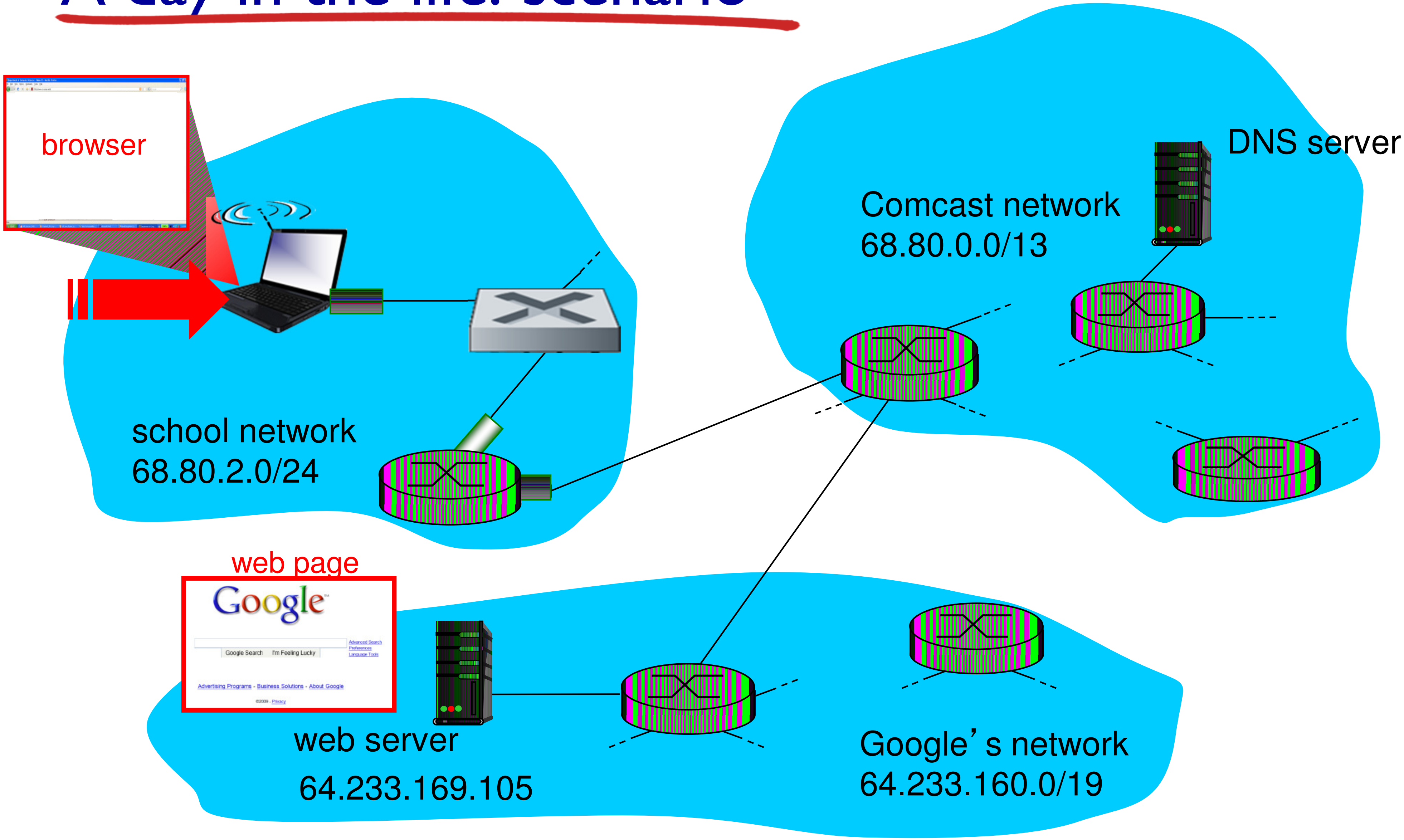
6.6 data center networking

6.7 a day in the life of a web request

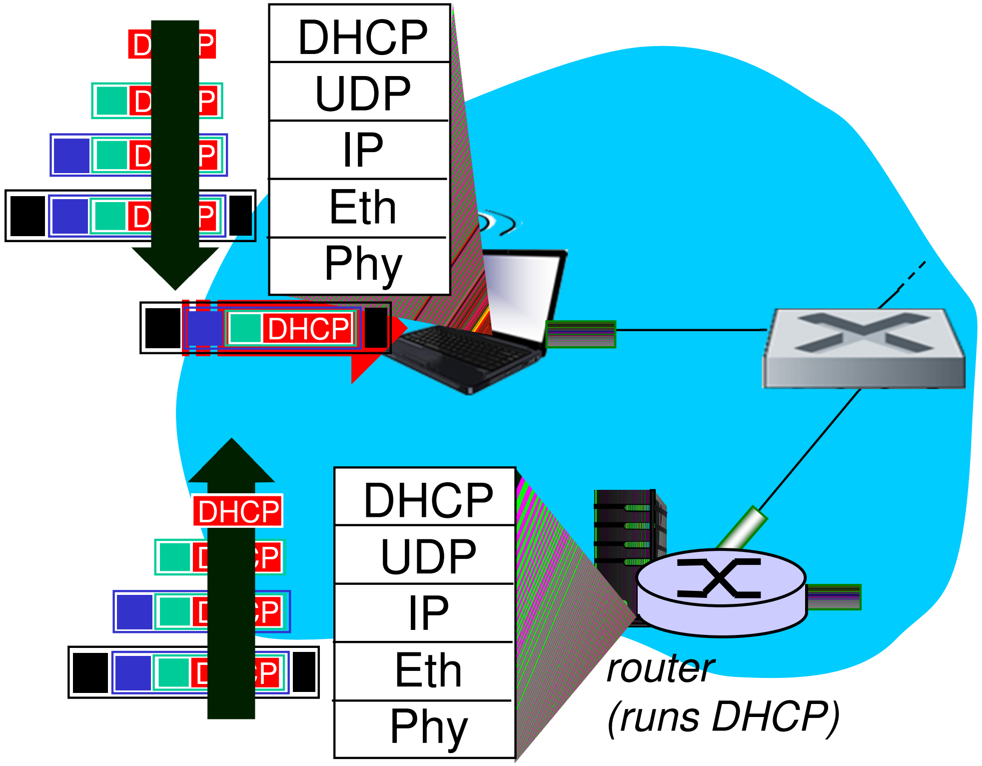
Synthesis: a day in the life of a web request

- journey down protocol stack complete!
  - application, transport, network, link
- putting-it-all-together: synthesis!
  - *goal:* identify, review, understand protocols (at all layers) involved in seemingly simple scenario: requesting www page
  - *scenario:* student attaches laptop to campus network, requests/receives www.google.com

A day in the life: scenario

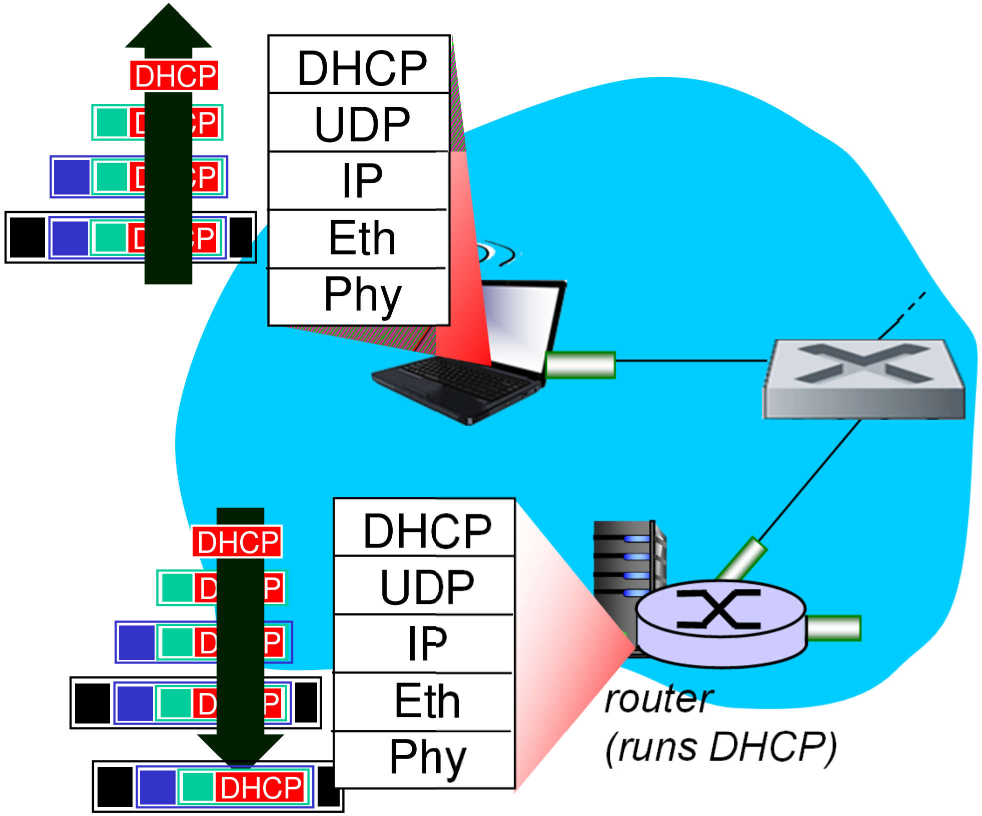


A day in the life... connecting to the Internet



- connecting laptop needs to get its own IP address, addr of first-hop router, addr of DNS server: use *DHCP*
- DHCP request *encapsulated* in *UDP*, encapsulated in *IP*, encapsulated in *802.3* Ethernet
- Ethernet frame *broadcast* (dest: FFFFFFFFFFFF) on LAN, received at router running *DHCP* server
- Ethernet *demuxed* to IP demuxed, UDP demuxed to DHCP

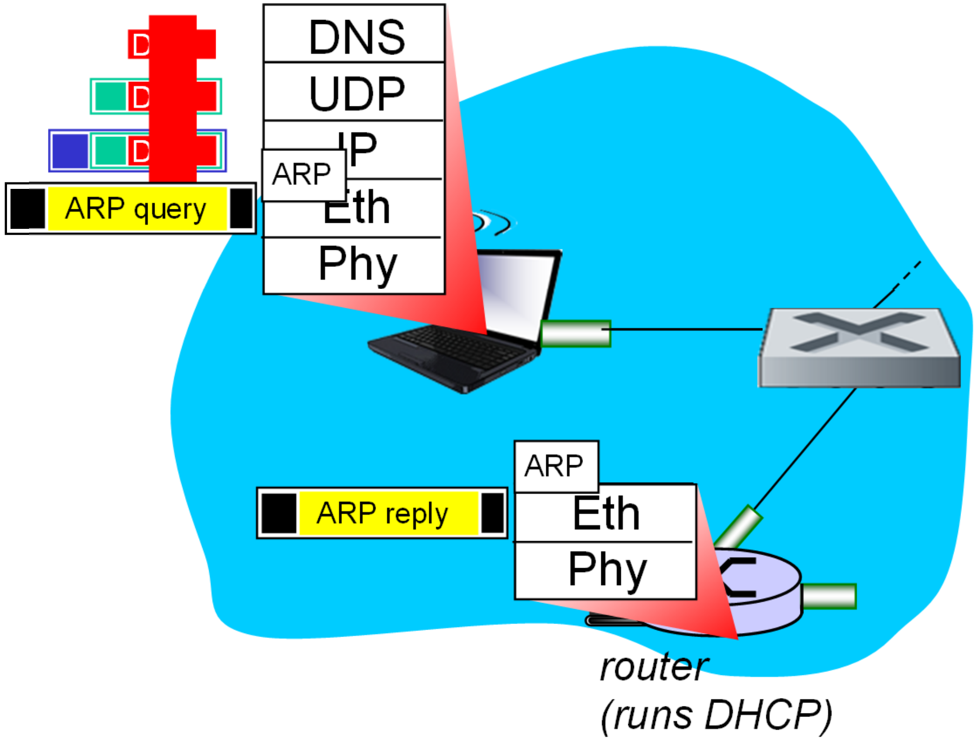
A day in the life… connecting to the Internet



- DHCP server formulates *DHCP ACK* containing client's IP address, IP address of first-hop router for client, name & IP address of DNS server
- encapsulation at DHCP server, frame forwarded (*switch learning*) through LAN, demultiplexing at client
- DHCP client receives DHCP ACK reply

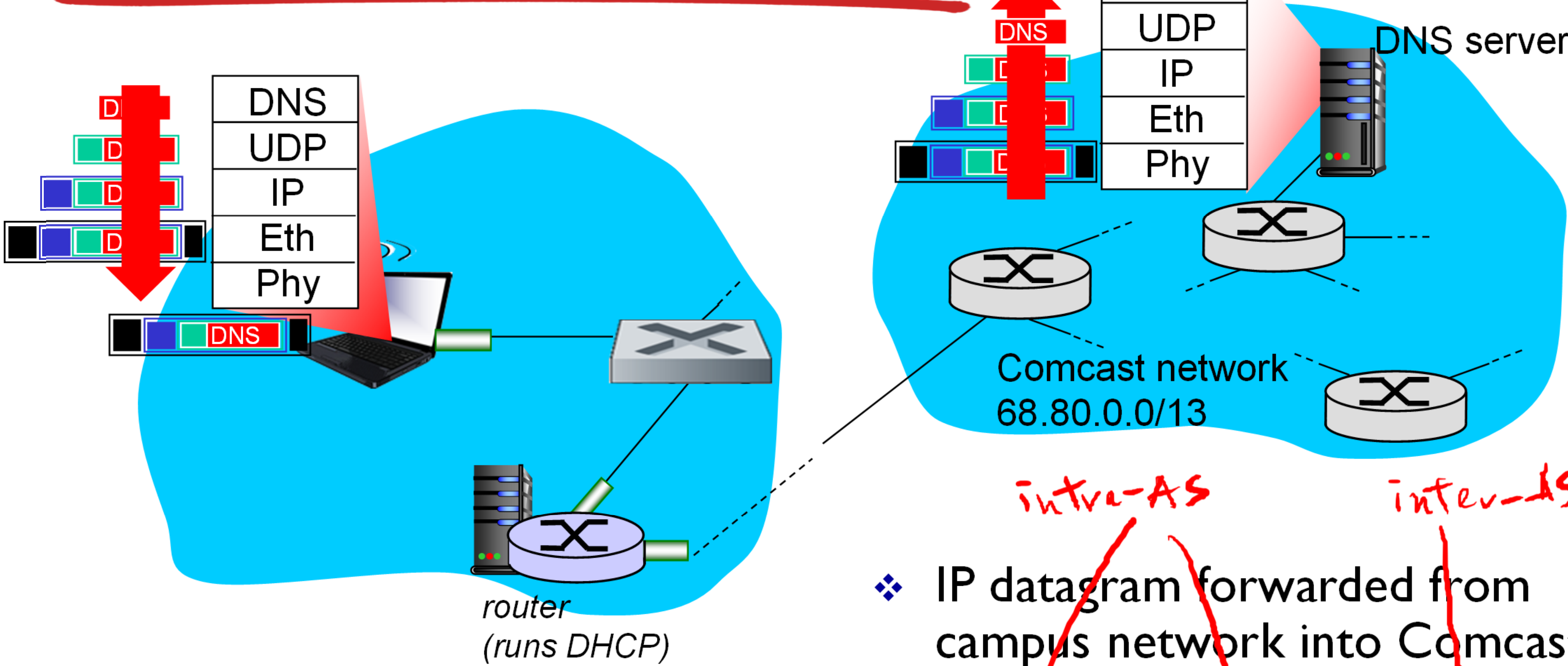
Client now has IP address, knows name & addr of DNS server, IP address of its first-hop router

A day in the life… ARP (before DNS, before HTTP)



- before sending *HTTP* request, need IP address of www.google.com: *DNS*
- DNS query created, encapsulated in UDP, encapsulated in IP, encapsulated in Eth. To send frame to router, need MAC address of router interface: *ARP*
- *ARP query* broadcast, received by router, which replies with *ARP reply* giving MAC address of router interface
- client now knows MAC address of first hop router, so can now send frame containing DNS query

A day in the life… using DNS



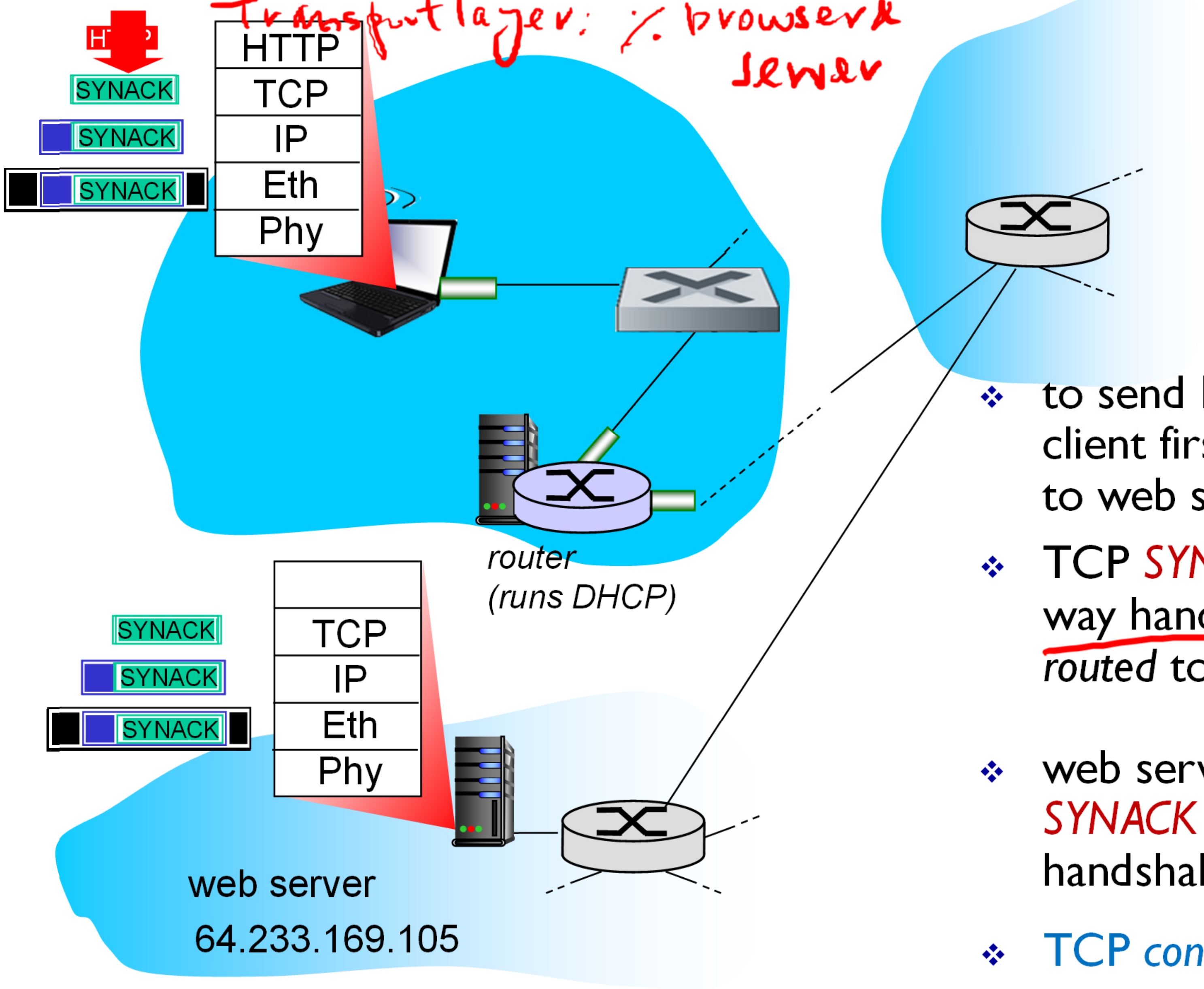
intra-AS inter-AS

- IP datagram containing DNS query forwarded via LAN switch from client to 1st hop router

- IP datagram forwarded from campus network into Comcast network, routed (tables created by *RIP, OSPF, IS-IS* and/or *BGP* routing protocols) to DNS server
- demux'ed to DNS server
- DNS server replies to client with IP address of www.google.com

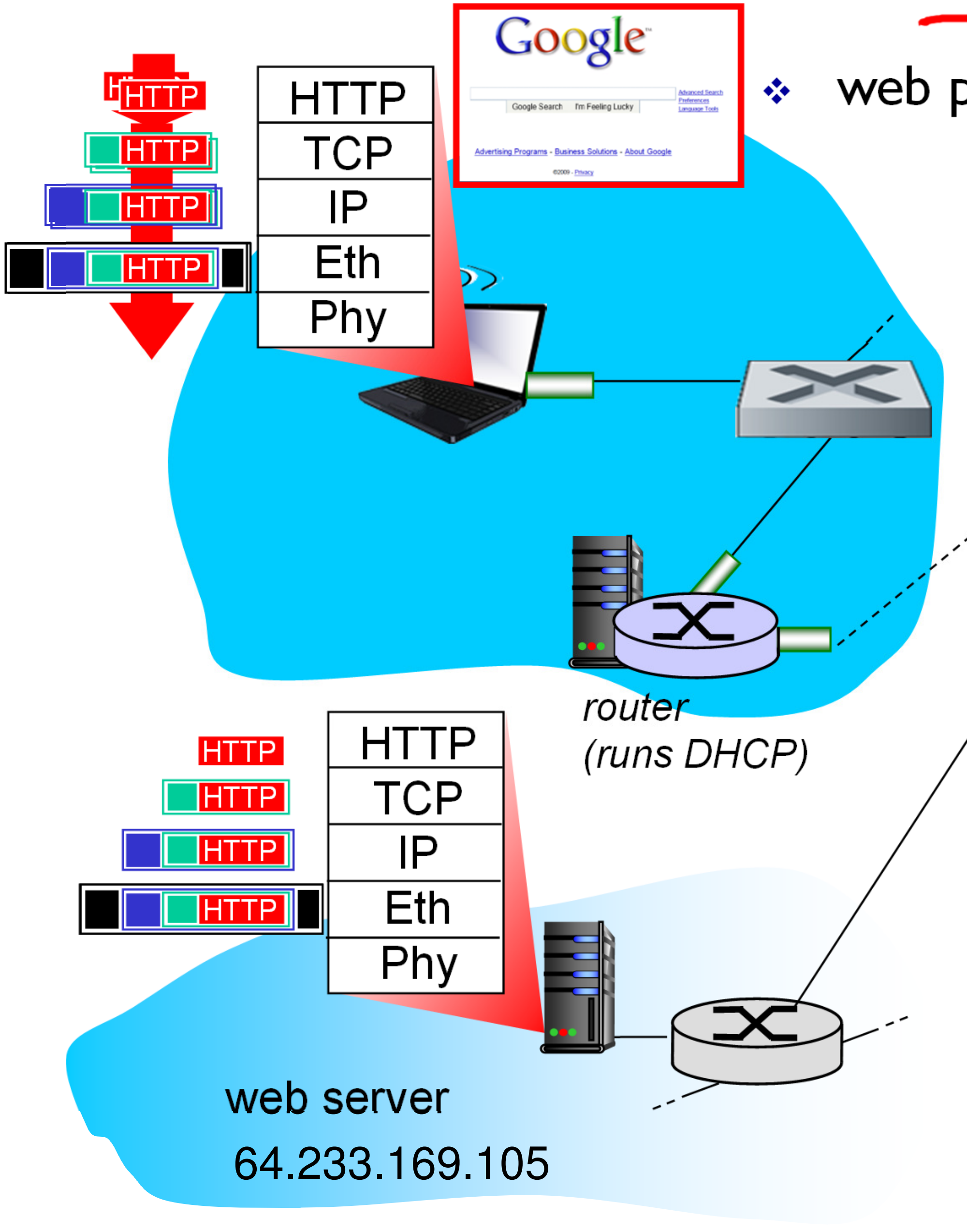
A day in the life… TCP connection carrying HTTP

Transport layer: browser & server



- to send HTTP request, client first opens *TCP socket* to web server
- TCP *SYN segment* (step 1 in 3-way handshake) *inter-domain routed* to web server
- web server responds with *TCP SYNACK* (step 2 in 3-way handshake)
- TCP *connection established!*

A day in the life… HTTP request/reply



- web page *finally (!!!)* displayed
- *HTTP request* sent into TCP socket
- IP datagram containing HTTP request routed to www.google.com
- web server responds with *HTTP reply* (containing web page)
- IP datagram containing HTTP reply routed back to client

Chapter 6: Summary

- principles behind data link layer services:
  - error detection, correction
  - sharing a broadcast channel: multiple access
  - link layer addressing
- instantiation and implementation of various link layer technologies
  - Ethernet
  - switched LANS, VLANs
  - virtualized networks as a link layer: MPLS
- synthesis: a day in the life of a web request

Chapter 6: let's take a breath

- journey down protocol stack *complete* (except PHY)
- solid understanding of networking principles, practice
- ….. could stop here …. but *lots* of interesting topics!
  - wireless
  - multimedia
  - security
  - network management